

Measuring Five-Nines Reliability: Sample-Efficient LLM Evaluation in Saturated Benchmarks

Eungyeup Kim Chenchen Gu* Vashisth Tiwari* J. Zico Kolter
Carnegie Mellon University

Abstract

While existing benchmarks demonstrate the near-perfect performance of large language models (LLMs) on various tasks, this apparent saturation often obscures the need for rigorous evaluation of their reliability. In real-world deployment, however, achieving extremely high reliability (e.g., “five-nines” (99.999%) vs. “three-nines” (99.9%)) is fundamentally critical, as this gap results in an order-of-magnitude increase in failures, which is catastrophic in reliability-critical applications. Still, estimating such a rare failure probability with tight confidence bounds requires prohibitively large LLM inference sizes, making standard Monte Carlo evaluation infeasible under limited compute budgets. In this paper, we observe that LLM failures exhibit strong systematic patterns: across broad parameterized input spaces, a small subset of inputs disproportionately accounts for the majority of failures. Leveraging this observation, we propose to learn a sampling distribution concentrated on failure-prone inputs via the cross-entropy method (CEM). We evaluate our framework on three LLMs, Qwen2.5-Math-7B-Instruct, gpt-oss-20b-low, and Gemini 2.5 Flash Lite, across parameterized GSM8K templates and achieve up to $156.22\times$ reduction in required inferences compared to naive uniform sampling. Our estimates reveal that models with indistinguishable accuracy on standard benchmarks can differ substantially in estimated failure rates, underscoring that reliability is a distinct and measurable axis of model quality. Our simple yet practical framework enables the evaluation of extreme reliability in LLMs, a distinct and underexplored dimension of evaluation beyond existing benchmarks, for their growing use in reliability-sensitive applications.

1 Introduction

Large language models (LLMs) show rapid progress not just in completing tasks with shorter duration (METR, 2026), but in giving saturating performance in numerous benchmarks (Maslej et al., 2025). For example, in AIME 2025 (American Invitational Mathematics Examination), GPT-5.2 Thinking (OpenAI, 2025) and Gemini-3-Pro (Google DeepMind, 2025) achieve a perfect score, while Claude Opus 4.6 (Anthropic, 2026) reaches 99.8%. While these near-perfect performances give the impression that models seem reliable at first glance, they may instead obscure how genuinely reliable these models are in real-world deployments where reliability becomes extremely important. As an example, in safety-critical settings such as refusing harmful requests (OpenAI, 2026; Anthropic, 2026), even a vanishingly small gap in accuracy, e.g., 99.999% (“five-nines”) and 99.9% (“three-nines”) in millions of queries, translates into an order-of-magnitude difference in failures, e.g., 1000 vs. 10 failures. Thus, this dimension of reliability evaluation is increasingly important but remains under-explored in current evaluation pipelines of existing benchmarks (Vendrow et al., 2024).

One major practical bottleneck is that, for such rare failures, achieving the tight confidence bounds requires a prohibitively large sample size via Monte Carlo sampling under a uniform

*co-second author, [Project page](#)

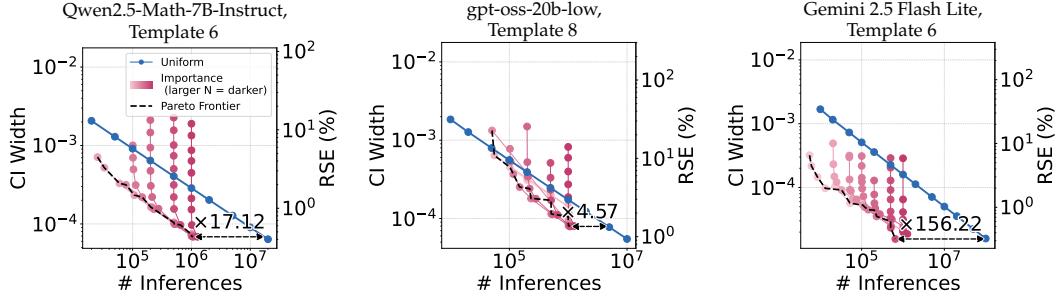


Figure 1: Each solid pink lines connects estimations with varying number of inferences (pink dots) under importance sampling with same number of overhead inferences for CEM (N), and darker pink means larger N . Similarly, solid blue line for uniform sampling. Only estimates with coverage within 1% tolerance of 99% are shown. The Pareto-frontier (dotted line) is the lower-left envelope across all (N , # evaluation) combinations, which achieves a tight CI width, i.e., relative standard error (RSE) of around 1%, at a much smaller inference budget compared to uniform sampling (e.g., $156.22\times$ gain for Gemini 2.5 Flash Lite on Template 6).

distribution P . For instance, to estimate a failure rate of $p = 10^{-5}$ (equally 99.999% accuracy) with small standard error ε , the binomial estimator $\sqrt{p(1-p)/n}$ requires $n \approx p/\varepsilon^2 = 10^{-5}/(10^{-6})^2 = 10^7$ when $\varepsilon = 10^{-6}$. Combined with the expense of LLM inference (which grows further with long reasoning traces in recent models), exhaustive uniform sampling for evaluation becomes computationally infeasible.

In this paper, we observe that *LLM failures are not uniformly distributed but exhibit systematic patterns*. Specifically, in parameterized GSM8K (Mirzadeh et al., 2025), across a combinatorially large input space, only a small set of inputs consistently induces failures at disproportionately high rates. For example, gpt-oss-20b-low (Agarwal et al., 2025) shows that approximately 82% of its failures strongly co-occur with the input prompts with a specific parameter value (e.g., fraction of blue ball = $2/7$ for GSM template 7), while other values rarely cause failures (Figure 2 middle row, Figure 3 right panel). Such concentration appears across different models, such as Qwen2.5-Math-7B-Instruct (Yang et al., 2024) and Gemini 2.5 Flash Lite (Comanici et al., 2025), and other templates in GSM. This phenomenon sharpens under self-consistency decoding with majority voting (Wang et al., 2023), which filters out random errors and reveals inputs that consistently trigger failures across multiple generations.

This systematic failure structure motivates our approach—rather than sampling uniformly, we concentrate *sampling on failure-prone regions*. We cast this as a rare-event estimation problem and adopt the CEM (Rubinstein, 1999; De Boer et al., 2005) to learn a new sampling distribution Q concentrated on failure-prone regions. From a small number of inferences, Q iteratively converges to a distribution that assigns higher probability to failure-prone input regions, which enables importance sampling for inference-efficient error estimation.

Our evaluation testbed includes multiple parameterized GSM8K templates (Cobbe et al., 2021; Mirzadeh et al., 2025), and three small but capable LLMs: Qwen2.5-Math-7B-Instruct, gpt-oss-20b-low, and Gemini 2.5 Flash Lite. Across models and templates, our method gains substantial inference efficiency, e.g., $156.22\times$ reduction in the required number of inferences for Gemini 2.5 Flash Lite on Template 6, for equivalent confidence bounds compared to uniform sampling (Figure 1 third plot). Notably, these gains grow as LLM failures become more concentrated, i.e., via increasing majority-vote ensemble size K . Specifically, efficiency improves from $3.68\times$ at $K = 4$ to $16.41\times$ at $K = 16$ for Qwen2.5-Math-7B-Instruct on Template 0 (Table 2). Our estimation results give $\sim 1\%$ relative standard errors for models with nearly 0.1% – 0.01% error rates, indicating that our method provides a practical evaluation pipeline for models’ reliability on extremely saturated tasks. Applying this pipeline across models, we find that models all exceeding 99.9% accuracy still differ by up to $2.4\times$ in estimated failure rates (e.g., gpt-oss-20b-low at 0.0254% vs. Qwen2.5-Math-7B-

Instruct at 0.0614% on Template 0, $K=16$), confirming that saturated benchmarks retain discriminative power at the reliability level (Figure 5).

2 Related Work

Evaluating reliability in saturated benchmarks. As LLMs push the frontier, they are evaluated on new, challenging benchmarks (Jimenez et al., 2024; Merrill et al., 2026; Rein et al., 2024; Yue et al., 2024; 2025; Center for AI Safety et al., 2026; Foundation, 2026), leaving behind older, saturated ones (Cobbe et al., 2021; Zellers et al., 2019; Wang et al., 2018; Sakaguchi et al., 2019) on which frontier models are now considered to perform nearly perfectly. Yet near-perfect is not perfect: Vendrow et al. (2024) showed that frontier LLMs, such as o1-2024-12-17 (high) (OpenAI, 2024), retain rare failures, around 0.1%, on GSM8K (Cobbe et al., 2021) even after correcting noisy label errors. Relatedly, GSM-Symbolic (Mirzadeh et al., 2025) demonstrates that frontier models’ near-perfect performance degrades on parameterized variants of the same problem templates. We focus on rigorous estimation of failure rates for models on highly saturated benchmarks, targeting regimes as extreme as “five-nines” (99.999%) accuracy, where the rarity of failures makes accurate estimation computationally costly compared to less saturated settings.

Efficient LLM evaluation. The autoregressive nature of LLMs makes inference expensive, and this cost grows as test-time compute scales (OpenAI, 2024; Guo et al., 2025). Several approaches reduce evaluation cost by selecting informative subsets of benchmark items (Maia Polo et al., 2024; Vivek et al., 2024; Perlitz et al., 2024; Hofmann et al., 2025), leveraging historical information (Wu et al., 2026), or using difficulty-adaptive testing (Truong et al., 2025). These studies target general capability evaluation on non-saturated benchmarks, whereas our work specifically addresses the rare-failure regime of highly saturated benchmarks.

Rare-event estimation with the CEM. The CEM (Rubinstein, 1999; Rubinstein & Kroese, 2004; De Boer et al., 2005) is a method for estimating rare-event probabilities by iteratively learning a proposal distribution concentrated on the rare-event region, enabling importance sampling with substantially reduced variance. A few works have adopted the CEM for rare-event estimation, including estimating failure probabilities of black-box physical systems in safety-critical applications (Arief et al., 2021) and estimating rare predictions of linear predictors (Bai et al., 2022). Our work is the first to apply the CEM to LLM evaluation, where the rare event is a majority-vote failure, the parameter space is a high-dimensional categorical distribution over problem instances, and each LLM inference is expensive.

3 Problem and Experimental Setup

Problem parameterization. To evaluate a model over a broad problem distribution, we construct a parameterized representation of task instances (e.g., mathematical reasoning prompts) as a mapping from the parameters Z to natural language prompts. Specifically, we define $Z = (Z_1, \dots, Z_d)$ as a vector of d independent categorical variables, where each Z_i takes values in a finite domain $\mathcal{Z}_i$. Following Mirzadeh et al. (2025), we parameterize all types of variables — including nouns and numerical values.

To illustrate, consider a template where specific nouns and values are treated as parameters:

Prompt (Template 0)

$\{Z_{\text{Name}}\}$ saw a $\{Z_x\}$ -foot $\{Z_{\text{Fish}}\}$ with $\{Z_y\}$ remoras, each $\{Z_z\}$ -inches long, attached to it. What percentage of the $\{Z_{\text{Fish}}\}$ ’s body length is the combined length of the remoras?

Each placeholder corresponds to a variable Z_i sampled from its domain $\mathcal{Z}_i$ (e.g., $Z_{\text{Name}} \in \mathcal{Z}_{\text{Name}}$, $Z_x \in \mathcal{Z}_x$). This formulation provides a structured, factorized representation of the problem input space. Let $\mathcal{Z} = \prod_{i=1}^d \mathcal{Z}_i$ denote the full parameter space. Details for templates are given in Appendix G.

Sampling distribution and failure. Given the parameter space $\mathcal{Z}$, we define a uniform sampling distribution:

$$P(Z) = \frac{1}{|\mathcal{Z}|}. \quad (1)$$

Our goal is to estimate the failure probability under P ,

$$\mu := \mathbb{E}_{Z \sim P} [\mathbb{P}(f(Z) = 1)], \quad (2)$$

where for each input Z , $f(Z) \in \{0, 1\}$ is a Bernoulli random variable indicating whether majority voting over K independent stochastic generations yields an incorrect prediction.

Modern LLMs often produce non-deterministic outputs (e.g., under non-zero temperature), especially for reasoning tasks. To obtain more consistent predictions, we adopt self-consistency decoding (Wang et al., 2023): for each input Z , we perform K independent inference runs and aggregate the outputs via majority voting. This reduces output variance by aggregating over diverse reasoning paths, approximating the mode of the model’s output distribution.

Formally, we define the failure indicator as

$$f(Z) := \mathbb{1}\{\text{MajorityVote}(\hat{y}_1(Z), \dots, \hat{y}_K(Z)) \neq y\}, \quad (3)$$

where $\hat{y}_k(Z)$ denotes the model output from the k -th independent inference on input Z , and y is the ground-truth label. Ties are counted as failures.

Datasets and models. Building on GSM-Symbolic (Mirzadeh et al., 2025), we generate approximately 100K instances per template by varying parameter ranges, yielding nine templates in total. Details of all templates used throughout our paper can be found in Appendix G. We notice that for some template-model pairs, failures are extremely rare (i.e., the template is too easy for some model) which is potentially non-zero but undetectable within our feasible evaluation budget. Therefore, we only report results for pairs with a sufficient number of observed failures to support reliable estimation. Details of such excluded pairs are provided in Section C.

Due to compute budget limits, we focus on relatively efficient yet capable LLMs, including Qwen2.5-Math-7B-Instruct (Yang et al., 2024), gpt-oss-20b-low (Agarwal et al., 2025) with low reasoning efforts, and Gemini 2.5 Flash Lite (Comanici et al., 2025). All inferences follow the models’ recommended decoding configurations (e.g., temperature, top- p , and max number of tokens), and details such as system prompts and parsing final answers are provided in Appendix A.

Bootstrapping Inferences. Ideally, evaluating an LLM requires fresh inferences for each generation, since LLM outputs are inherently stochastic (i.e., non-zero temperature), particularly during reasoning. However, this becomes computationally infeasible for evaluating with large sample sizes for rare failures, across multiple models, templates, and random seeds (for coverage). Therefore, we employ *bootstrap resampling* (Efron, 1979) to approximate the stochastic variation of LLM outputs without fresh inference for each input prompt. Specifically, we pre-compute and cache a pool of independent generations for each input, 16 generations for Qwen2.5-Math-7B-Instruct, 40 for gpt-oss-20b-low, and 32 for Gemini 2.5 Flash Lite, which is deemed large enough to approximate the true sampling. We randomly sample K outputs with replacement from this cached pool to simulate independent generations for majority voting. This resampling approximates the distribution of majority-vote outcomes given fresh inferences without additional inference costs.

4 LLM Failures Are Systematic, Not Random

Failures concentrate on a small subset of parameters. We observe that *LLM failures are not random, but exhibit systematic structure*. Within the high-dimensional parameter space $\mathcal{Z}$, a small subset of parameter values disproportionately accounts for a large fraction of

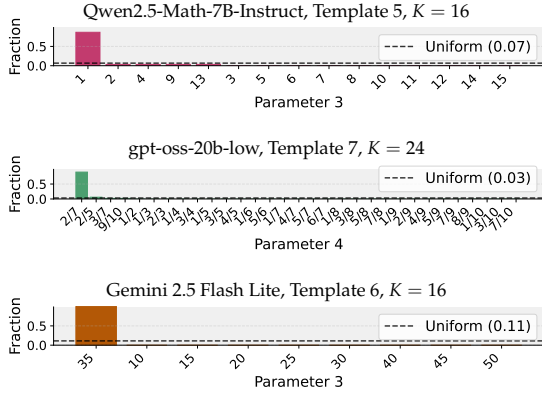


Figure 2: Distribution of failures across parameter values for model–template pairs. Each bar represents the proportion of failures for each parameter value, and the dotted line shows the expected uniform error rate.

Prompt (Template 5)

{Katy} makes {coffee} using teaspoons of sugar and cups of water in the ratio of {15}: {1}. If she used a total of {736} teaspoons of sugar and cups of water, calculate the number of teaspoonfuls of sugar she used.

LLM Output (Qwen2.5-Math-7B-Instruct)

To determine the number of teaspoonfuls of sugar Katy used, we start by

...

Since 1 cup of water is equivalent to 16 teaspoons (assuming standard US measurements

...

Therefore, the number of teaspoonfuls of sugar Katy used is: 360
 Answer: 690

Prompt (Template 7)

{Jamie} is juggling at a {circus}. {Jamie} can juggle {168} balls. {1/2} of the balls are {golf} balls, and {2/7} of the {golf} balls are {blue}. How many {blue} {golf} balls are there?

LLM Output (gpt-oss-20b-low)

analysisCompute: total balls 168.
 Half are golf balls: 168/2=84
 golf balls. Of those, 2/7 are blue:
 84*(2/7)=12.assistantfinalJamie has
 (12) blue golf balls.
 Answer: 24

Figure 3: Example pairs of prompt–generations that illustrate repetitive failures by LLMs. Failure-inducing parameter values and output traces are highlighted in red.

failures. Across models and templates, we observe striking concentration of failures on individual parameter values (Figure 2). We first iterate over all possible inputs for testing models, and aggregate all failures observed. We observe that a single parameter value accounts for the majority of errors: parameter term of water cups = 1 in Template 5 for Qwen2.5-Math-7B-Instruct (86% of failures), parameter fraction of blue ball = 2/7 in Template 7 for gpt-oss-20b-low (Agarwal et al., 2025) (82%), and parameter area of fruits = 35 in Template 6 for Gemini 2.5 Flash Lite (Comanici et al., 2025) (81%). Bar plots for all parameters are given in Appendix Figure 6.

To quantify this concentration, we compute the total variation (TV) distance between the normalized histogram of the failure counts over each parameter dimensions (Z_i such as Z_{Name}) and the uniform distribution (Table 1). The uniform distribution represents a baseline where the failures occur evenly across parameters. Thus, larger distances indicate more pronounced failure concentration, while a value of zero indicates that failures are spread evenly. For each setup, we report the top-3 parameter dimensions with the largest divergence. Across models and templates, we observe consistently large deviations from the uniform distribution, i.e., most Top-1 distances reach 0.9. Furthermore, as we increase

Model	ID	Distance		
		Top1	Top2	Top3
Qwen2.5-Math-7B-Instruct (K = 16)	0	0.900	0.750	0.509
	1	0.914	0.833	0.750
	2	0.538	0.249	0.019
	3	0.472	0.362	0.310
	4	0.339	0.264	0.185
	5	0.940	0.867	0.800
	6	0.879	0.752	0.247
	7	0.798	0.748	0.557
gpt-oss-20b-low (K = 24)	0	0.933	0.850	0.729
	2	0.699	0.541	0.052
	4	0.631	0.508	0.385
	7	0.895	0.633	0.537
	8	0.760	0.599	0.487
Gemini 2.5 Flash Lite (K = 16)	0	0.867	0.850	0.267
	2	0.907	0.750	0.080
	4	0.208	0.133	0.102
	6	0.983	0.889	0.189

Table 1: Total variation distance between failure distributions and uniform baseline. Parameters with the three largest distances are reported for each template.

K , we observe that the TV distance increases (Table 3), which implies that with larger K , the majority of failures becomes more pronounced, with fewer inconsistent failures. Specifically, larger K filters out random errors—failures caused by incidental factors in individual generations—while amplifying systematic failure patterns that persist across multiple samples. These results suggest that LLM failures are not spread randomly across the parameter space, but are instead driven by a small subset of failure-inducing inputs.

Qualitative analysis of repetitive failures. To understand how the failures concentrate in certain regions of the parameter space, we conduct a qualitative analysis of LLM-generated outputs that exhibit repetitive failures, as in Figure 3. We mark in red the input parameter values that frequently induce failures as well as the erroneous reasoning trace. For instance, Qwen2.5-Math-7B-Instruct consistently applies an unjustified unit conversion heuristic when the parameter value term for cup of water = 1 is given in Template 5. gpt-oss-20b-low fails at simple multiplication when 2/7 is given as fraction of blue ball in Template 7. Similarly, Gemini 2.5 Flash Lite fails in 35×43 in Template 6 (Figure 27). These models exhibit such errors repeatedly across multiple generations, as illustrated in Figures 19, 23, and 27. We provide additional qualitative analysis for other template-model pairs in Section I, which show various error types including wrong arithmetic operations, unnecessary approximation, problem misinterpretation, and others. These examples reveal that LLM errors in GSM problems stem from consistent, input-dependent reasoning behaviors, providing insight into the conditions under which models are likely to produce incorrect reasoning trajectories. Despite there being no a priori reason to expect systematic failure patterns to persist across template-model pairs, we find compelling empirical evidence that such patterns arise consistently across most pairs we examine.

5 Learning Failure-Prone Sampling Distributions

Our key finding of the concentration of failures in Section 4 naturally leads to a failure-prone sampling strategy—rather than drawing samples from the full parameter space under P , we concentrate sampling on failure-prone regions. To this end, we leverage the CEM to iteratively refine a proposal distribution Q toward regions of the parameter space where failures are more likely, which we can use to estimate true error probability via importance sampling.

Importance Sampling with Q . To estimate μ defined in Eq. 2, a standard Monte Carlo estimator under P is

$$\hat{\mu}_P = \frac{1}{N} \sum_{i=1}^N f(Z_i), \quad Z_i \sim P.$$

However, when failures are rare, the estimator suffers from high variance relative to μ , requiring large numbers of samples to obtain tight confidence bounds. To improve efficiency, we introduce a proposal distribution Q and estimate μ via importance sampling:

$$\hat{\mu}_Q = \frac{1}{N} \sum_{i=1}^N f(Z_i) \frac{P(Z_i)}{Q(Z_i)}, \quad Z_i \sim Q,$$

which yields an unbiased estimate under the support condition that $Q(Z) > 0$ whenever $\mathbb{P}(f(Z) = 1)P(Z) > 0$. To ensure that importance weights are bounded, we employ defensive sampling (Hesterberg, 1995), which mixes Q with P with mixing coefficient λ .

CEM for learning Q . The variance-minimizing proposal distribution is given by $Q^*(Z) = \frac{\mathbb{P}(f(Z)=1)P(Z)}{\mu}$, which concentrates probability mass on failure regions. The CEM seeks a proposal distribution Q that approximates this optimal distribution by minimizing the Kullback-Leibler divergence, $\text{KL}(Q^* \parallel Q)$. Since Q^* depends on the unknown μ , the CEM iteratively refines Q by sampling from the current proposal and updating toward the empirical failure-weighted distribution proportional to $\mathbb{P}(f(Z) = 1)P(Z)$.

Algorithm 1 Failure Probability Estimation via CEM

```

1: Input: Failure indicator  $f$ , original distribution  $P$ , sample size  $N$ , smoothing coefficient  $\zeta$ , iterations  $T$ , confidence level  $\alpha$ , defensive sampling coefficient  $\lambda$ 
2: Output: Failure probability estimate  $\hat{\mu}$ , confidence interval  $[\hat{\mu}_{\text{low}}, \hat{\mu}_{\text{high}}]$ 
3: // CEM for updating  $Q$ 
4: Initialize  $Q_0(Z_i)$  as uniform over  $\mathcal{Z}_i$  for all  $i = 1, \dots, d$ 
5: for  $t = 1$  to  $T$  do
6:   // Sampling
7:   Sample  $Z_1, \dots, Z_N \sim Q_t(Z) = \prod_{i=1}^d Q_t(Z_i)$ 
8:   // Elite set selection
9:   Define elite set  $\mathcal{E}_t \leftarrow \{n : f(Z_n) = 1\}$ 
10:  if  $\mathcal{E}_t = \emptyset$  then
11:    Set  $Q_{t+1} \leftarrow Q_t$  and continue
12:  end if
13:  // Weighted update
14:  for each dimension  $i$  and value  $j \in \mathcal{Z}_i$  do
15:    Compute importance weights  $w(Z_n) \leftarrow P(Z_n) / Q_t(Z_n)$ 
16:     $Q'_{t+1}(Z_i)_j \leftarrow \frac{\sum_{n \in \mathcal{E}_t} w(Z_n) \mathbb{1}_{\{Z_{n,i}=j\}}}{\sum_{n \in \mathcal{E}_t} w(Z_n)}$ 
17:  end for
18:  // Smoothing
19:   $Q_{t+1} \leftarrow \zeta Q'_{t+1} + (1 - \zeta) Q_t$ 
20: end for
21: Apply defensive sampling  $\tilde{Q}_T(Z) \leftarrow (1 - \lambda) Q_T(Z) + \lambda P(Z)$ 
22: // Estimation with confidence bounds
23:  $\hat{\mu}, [\hat{\mu}_{\text{low}}, \hat{\mu}_{\text{high}}] \leftarrow \text{COMPUTEESTIMATE}(f, P, \tilde{Q}_T, N, \alpha)$ 
24: Return  $\hat{\mu}, [\hat{\mu}_{\text{low}}, \hat{\mu}_{\text{high}}]$ 

```

Failure probability estimation via Q learned from CEM. To begin with, we model Q as a fully factorized distribution:

$$Q(Z) = \prod_{i=1}^d Q(Z_i), \quad (4)$$

where each $Q(Z_i)$ is a categorical distribution over $\mathcal{Z}_i$. This independence assumption provides a tractable approximation to model the joint parameter space. We initialize $Q_0(Z)$ as a uniform distribution over each parameter dimension $Z_i \in \mathcal{Z}_i$.

We then apply the CEM followed by importance sampling for estimating true failure probability with Q , as described in Algorithms 1 and 2.

6 Order-of-Magnitude Inference Efficiency Gains

Setup for CEM For CEM, we use fixed hyperparameters across all models and templates: $T=10$ iterations, smoothing coefficient $\zeta=0.1$, and defensive sampling coefficient $\lambda \in \{0.1, 0.3, 0.5\}$. The CEM sample size N , the number of data for updating Q at each CEM iteration t , is set as a hyperparameter, ranging between 0.5K and 100K.

Setup for estimation. For the final estimation, we vary the evaluation sample size M between 1K and 1M (million). We define the *number of inferences* as the number of samples from P or Q (one inference per Z , independent of K). Because the $T \times N$ samples drawn during CEM can be reused directly as importance sampling estimation samples, the total number of inferences under Q is $\max(T \times N, M)$: if the CEM draws already reach or exceed M , no additional inferences are needed; otherwise, $M - T \times N$ samples are drawn from $\tilde{Q}_T$. For P , the cost is simply M inferences with no CEM phase.

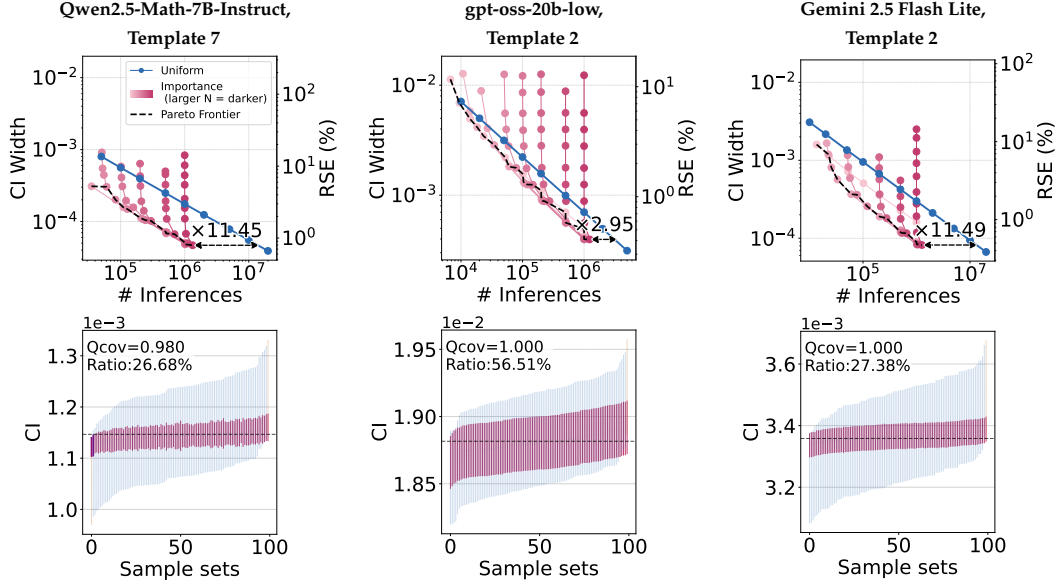


Figure 4: **Upper row: CI width vs. number of inferences.** Each solid pink lines connects estimations with varying number of inferences (pink dots) under importance sampling with same number of overhead inferences for CEM (N), and darker pink means larger N . Similarly, solid blue line for uniform sampling. The Pareto-frontier (dotted line) is the lower-left envelope across all $(N, \# \text{ evaluation})$ combinations, which achieves a tight CI width, i.e., relative standard error (RSE) of around 1%, at a much smaller inference budget compared to uniform sampling. **Lower row: sorted CIs over 100 sample sets.** At the evaluation size of the bottom-right Pareto-frontier dot, Q (pink) yields much narrower confidence bounds than P (blue) for the same sample size, while maintaining coverage close to 99%. We highlight the intervals that fail to cover the population mean in dark pink.

Setup for evaluation. For all evaluations, we construct 99% confidence intervals (significance level $\alpha = 0.01$) using the normal approximation applied to the importance-weighted estimator $\hat{\mu}_Q$. To verify the unbiasedness of our estimator, we draw 100 independent sample sets for each sample size and report the estimates whose coverage falls within $99 \pm 1\%$ (i.e., 1% tolerance of the nominal coverage level). For calculating coverage, we use $\hat{\mu}_P$ computed using 1B bootstrap samples from P , which converges to the unknown true mean.

We compare against uniform sampling from P with confidence bounds computed via the Clopper-Pearson binomial exact method (Clopper & Pearson, 1934). To quantify the inference efficiency obtained by importance sampling, we present the *sampling efficiency gain*, calculated by the number of inferences under P divided by that under Q (i.e., $\max(T \times N, M)$) to obtain the same CI width. Across N and evaluation sizes, we report the gains of the Pareto-frontier in the CI width vs. number of inferences trade-off (i.e., dotted line in Figures 1 and 4). We additionally report RSE defined as $RSE = \frac{SE}{\hat{\mu}}$ to evaluate the tightness of the confidence bounds.

Failure-prone sampling saves an order of magnitude in inferences. Our results demonstrate that the learned distribution Q , which concentrates on frequent failure patterns, yields substantially tighter confidence bounds compared to uniform sampling under P . In Figure 1 and upper row of Figure 4, we present CI width under Q (pink) for varying evaluation sizes (dots connected by lines) with different N , the number of samples for CEM (different shades). The Pareto-frontier (dotted line), which connects the minimum number of inferences required to achieve each CI width, decreases much more rapidly under Q as the number of inferences increases than under P (blue).

This indicates that, for a tight confidence bound such as an RSE of 1%, importance sampling under Q requires significantly fewer inferences than uniform sampling from P . Specifically,

	$\lambda = 0.1$						$\lambda = 0.3$						$\lambda = 0.5$					
	Gains			RSE (%)			Gains			RSE (%)			Gains			RSE (%)		
Qwen2.5-Math-7B-Instruct																		
ID	K=4	K=8	K=16	K=4	K=8	K=16	K=4	K=8	K=16	K=4	K=8	K=16	K=4	K=8	K=16	K=4	K=8	K=16
0	3.68	6.50	16.41	1.24	1.22	1.02	4.51	8.03	15.84	1.27	1.21	1.05	3.82	6.59	13.65	1.27	1.26	1.13
1	1.31	2.29	5.33	1.97	2.99	2.95	1.58	2.34	7.02	1.94	3.00	2.46	1.57	2.30	6.02	1.95	3.21	2.69
2	1.63	1.48	1.87	0.22	0.24	0.27	1.37	1.48	1.80	0.22	0.25	0.27	1.51	1.61	1.68	0.22	0.26	0.29
3	1.07	1.46	1.75	0.43	0.63	0.80	1.25	1.50	1.49	0.43	0.63	0.81	1.24	1.25	1.64	0.43	0.64	0.83
4	1.34	1.38	1.22	0.31	0.36	0.40	1.30	1.38	1.23	0.31	0.36	0.40	1.29	1.33	1.41	0.31	0.37	0.41
5	2.15	3.67	6.93	1.20	1.77	5.05	2.51	4.28	5.40	1.14	1.87	2.81	2.32	3.69	5.62	1.19	1.85	2.78
6	5.36	15.84	17.12	0.44	0.44	0.43	6.40	12.87	15.92	0.49	0.49	0.47	5.80	9.46	11.74	0.56	0.56	0.54
7	3.22	8.02	11.45	1.08	0.95	0.79	3.53	8.45	14.66	1.03	0.89	0.80	4.44	6.42	11.59	1.03	0.96	0.87
8	1.91	—	—	4.97	—	—	2.73	—	—	4.09	—	—	2.45	—	—	4.57	—	—
gpt-oss-20b-low																		
ID	K=8	K=16	K=24	K=8	K=16	K=24	K=8	K=16	K=24	K=8	K=16	K=24	K=8	K=16	K=24	K=8	K=16	K=24
0	3.34	3.81	3.63	2.56	2.95	3.92	2.95	3.79	6.77	2.52	3.10	2.85	2.71	4.45	3.54	2.63	3.11	4.15
2	2.27	2.70	2.95	0.28	0.37	0.41	2.28	2.86	3.28	0.30	0.39	0.43	1.93	2.35	2.81	0.32	0.42	0.46
4	1.30	1.93	2.65	0.79	1.19	1.36	1.55	1.69	2.30	0.79	1.16	1.34	1.53	1.91	2.09	0.80	1.19	1.40
7	2.89	2.80	4.45	1.61	2.76	2.29	3.07	3.50	5.22	1.62	2.14	2.18	2.39	4.29	5.39	1.65	2.11	2.31
8	4.36	4.41	4.57	0.84	1.18	1.36	4.02	4.42	4.65	0.90	1.21	1.42	2.99	3.43	3.70	0.97	1.32	1.49
Gemini 2.5 Flash Lite																		
ID	K=4	K=8	K=16	K=4	K=8	K=16	K=4	K=8	K=16	K=4	K=8	K=16	K=4	K=8	K=16	K=4	K=8	K=16
0	0.96	4.05	5.29	1.95	1.90	2.35	1.23	4.74	8.23	1.64	1.72	1.74	1.35	4.90	8.62	1.57	1.71	1.93
2	2.54	4.41	11.49	0.68	0.62	0.47	2.76	5.69	10.35	0.65	0.62	0.51	2.38	5.16	9.50	0.66	0.64	0.57
4	1.12	0.99	1.00	0.40	0.51	0.58	1.11	1.18	1.19	0.40	0.51	0.58	0.95	1.16	1.00	0.40	0.51	0.58
6	1.06	2.75	156.22	0.99	1.80	0.32	1.10	3.45	85.21	0.98	1.63	0.36	1.27	3.32	82.59	0.98	1.52	0.37

Table 2: Sampling efficiency gains and RSE (%) across templates, models, K , and λ for majority voting. Cell shading for gains is proportional to the gain value, i.e., the higher the darker, capped at gain ≥ 10 . — denotes zero failures.

such inference efficiency gains span orders of magnitude across models and templates, e.g., $17.12\times$ for Qwen2.5-Math-7B-Instruct on Template 6 ($K = 16$), $4.57\times$ for gpt-oss-20b-low on Template 8 ($K = 24$), and $156.22\times$ for Gemini 2.5 Flash Lite on Template 6 ($K = 16$).

These inference efficiency gains stem from the tight CIs obtained through importance sampling, as illustrated in Figure 4 bottom row. The plots compare the confidence bounds between P and Q across 100 independent sample sets (sorted by their center). Across evaluation sizes, we plot the bounds for the sample size corresponding to the bottom-right pink dot on the Pareto-frontier. Intervals under Q (pink) are much narrower than those under P (blue) for the same sample size. These 100 intervals faithfully bound the $\hat{\mu}_P$ (with ± 1 tolerance), confirming that our method meets the nominal 99% confidence level.

We report inference efficiency gains and their RSE across models and templates in Table 2. For each gain, we add darker shading for larger values for visibility. The results demonstrate that our method achieves consistent efficiency gains, where most exceed $2\times$. These estimations show substantially low RSE, where most are around just 1%, implying that we can estimate LLMs’ true error probability with highly accurate estimation across templates and models.

We observe that inference efficiency gains improve as we use larger K for self-consistency decoding (Wang et al., 2023), i.e., shading becomes darker as K increases in gains. For instance, Qwen2.5-Math-7B-Instruct shows a gain of $3.68\times$ with $K = 4$ increasing to $16.41\times$ with $K = 16$ on Template 0, sharing the trends across setups. These results corroborate our empirical observations that systematic failures become more concentrated with higher K . In addition, overall gains and RSE remain consistent across different λ , which allows practitioners to use our method without careful hyperparameter tuning.

Similarly, efficiency gains are strongly correlated with failure rarity: as models become more saturated on a task, failures concentrate on a smaller subset of inputs, making importance sampling more effective. As a concrete example, Gemini 2.5 Flash Lite shows higher error rate ($\hat{\mu}_Q$) on Template 4, i.e., 2.59%, than others (0.335%-0.0328%) with $K = 16$ correspondingly, those high error-rate templates also exhibit lower TV distance (Table 1)

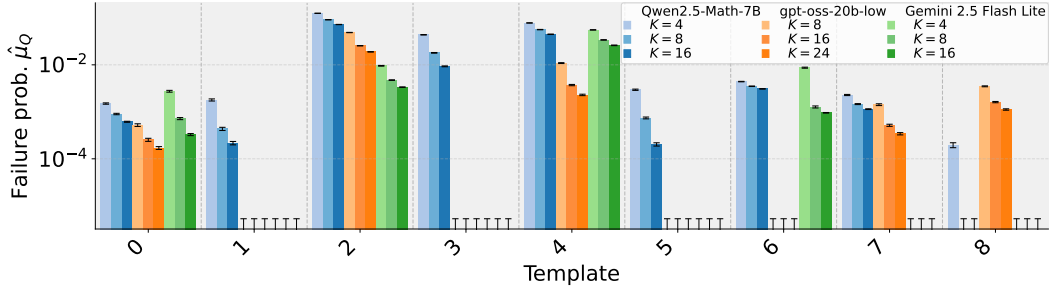


Figure 5: Estimated failure probability $\hat{\mu}_Q$ across templates and models at $\lambda = 0.1$ and evaluation size of $1M$, with error bars denoting the half-width of the confidence interval. For those with zero failures, we report the error bars of exact binomial bounds, i.e., $(0, 5.30e-06)$.

and smaller efficiency gains (Table 2), which is consistent with our finding that stronger failure concentration drives higher importance sampling efficiency.

7 Reliability Evaluation Between Models on GSM

Finally, Figure 5 reports each model’s estimated rare failure probability $\hat{\mu}_Q$ across GSM templates, with evaluation size of $M = 1M$, presented as a point estimate with confidence bounds. While all models achieve saturating performance with error rates well below 1%, our evaluation reveals meaningful differences in reliability at finer granularity. Specifically, gpt-oss-20b-low gives a failure rate of just 0.0254% on Template 0 with $K = 16$, which outperforms Qwen2.5-Math-7B-Instruct (0.0614%, $\sim 2.5\times$ higher) and Gemini 2.5 Flash Lite (0.0328%, $\sim 1.3\times$ higher). These results highlight that such saturated benchmarks still carry discriminative signal for reliability, and such evaluation encourages the community to pursue models that are not only more capable but more reliably so. For all template-model pairs that include zero failures, we report the exact binomial bounds, i.e., $(0, 0.00053\%)$. The full table is provided in Table 4.

8 Limitations, Future Work, and Conclusion

Limitations. Our method relies on the assumption that LLM errors concentrate on a small subset of the parameter space, which we validate empirically across the models and templates we study. When error patterns are diffuse, e.g., when the model is relatively weak for a task, the CEM may not find a substantially better proposal than uniform sampling. Our method may also be less effective when estimating failure rates without majority voting ($K=1$), where stochastic generation outputs produce inconsistent failure patterns that are harder to concentrate on.

In addition, due to computational constraints, our experiments focus on relatively small LLMs. Evaluating reliability introduces significant overhead: the required number of inferences grows with the evaluation sample size, the majority-vote ensemble size K , and the number of independent trials for coverage verification, all of which increase as failures become rarer or as the desired confidence level tightens. Extending this framework to larger, more capable LLMs therefore remains an important direction for future work.

Lack of reliability benchmarks. While nearly all existing benchmarks focus on evaluating how LLMs push the frontier in challenging tasks, to our knowledge, no benchmark specifically tests their reliability on already-saturated benchmarks. Rather than revisiting these saturated benchmarks, the community tends to curate novel, more challenging ones, leaving a gap in our understanding of model reliability. In this paper, we take a first step toward addressing this gap by introducing a prototypical framework for evaluating LLM reliability using parameterized GSM templates. However, reliability evaluation must be extended to broader domains, including safety guardrails, code generation for production

systems, and medical or legal question answering, where even rare failures can have serious consequences. For instance, GPT-5.2-Thinking achieves a 99.9441% no-violation rate on internal safety benchmarks (OpenAI, 2026), yet the remaining 0.0559% violation rate still corresponds to over 500 failures per million queries, underscoring the importance of rigorous reliability measurement even in near-perfect regimes.

Surgical fixes for systematic failures. Identifying model failures that are consistently triggered by certain input prompts could enable more targeted reliability improvements. Our method directly supports this: the learned proposal distribution Q highlights the failure-inducing parameter values, which practitioners can use as targeted training data or test cases to surgically correct systematic failures via fine-tuning on problematic inputs or adding safeguards rather than retraining broadly. The ultimate goal is to leverage these systematic failure patterns to push model reliability into the five-nines regime, enabling safe deployment in high-stakes applications.

Conclusion. We present an inference-efficient framework for estimating the true error probability of highly accurate LLMs, demonstrated on parameterized GSM problems. LLM failures are strongly concentrated on a small subset of inputs, as evidenced by high total variation distances from the uniform distribution across all models and templates we study. Leveraging this finding, we adopt the CEM to learn a failure-prone sampling distribution, achieving up to $156.22\times$ inference savings over uniform sampling for equivalent confidence bounds. We call on the community to treat reliability as a first-class evaluation criterion alongside capability; our framework provides a practical foundation for rigorous reliability measurement in high-stakes LLM deployments.

9 Acknowledgements

We thank Yixuan Xu, Bingqing Chen, and Chen Qiu for insightful discussions. Eungyeup Kim is supported by funding from Bosch Center for Artificial Intelligence. Vashisth Tiwari is funded by the National Institute of Standards and Technology under Federal Award ID Number 60NANB24D231 and Carnegie Mellon University AI Measurement Science and Engineering Center (AIMSEC).

References

- Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- Anthropic. Claude opus 4.6 system card, 2026. URL <https://anthropic.com/claude-opus-4-6-system-card>.
- Mansur Arief, Zhiyuan Huang, Guru Koushik Senthil Kumar, Yuanlu Bai, Shengyi He, Wenhao Ding, Henry Lam, and Ding Zhao. Deep probabilistic accelerated evaluation: A robust certifiable rare-event simulation methodology for black-box safety-critical systems. In Arindam Banerjee and Kenji Fukumizu (eds.), *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*, volume 130 of *Proceedings of Machine Learning Research*, pp. 595–603. PMLR, 13–15 Apr 2021. URL <https://proceedings.mlr.press/v130/arief21a.html>.
- Yuanlu Bai, Zhiyuan Huang, Henry Lam, and Ding Zhao. Rare-event simulation for neural network and random forest predictors. *ACM Trans. Model. Comput. Simul.*, 32(3), July 2022. ISSN 1049-3301. doi: 10.1145/3519385. URL <https://doi.org/10.1145/3519385>.
- Center for AI Safety, Scale AI, and HLE Contributors Consortium. A benchmark of expert-level academic questions to assess AI capabilities. *Nature*, 649:1139–1146, 2026. doi: 10.1038/s41586-025-09962-4. URL <https://arxiv.org/abs/2501.14249>.

- C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. ISSN 00063444, 14643510. URL <http://www.jstor.org/stable/2331986>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025. URL <https://arxiv.org/abs/2507.06261>.
- Pieter-Tjerk De Boer, Dirk P Kroese, Shie Mannor, and Reuven Y Rubinstein. A tutorial on the cross-entropy method. *Annals of operations research*, 134(1):19–67, 2005.
- B. Efron. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1):1 – 26, 1979. doi: 10.1214/aos/1176344552. URL <https://doi.org/10.1214/aos/1176344552>.
- ARC Prize Foundation. Arc-agi-3: A new challenge for frontier agentic intelligence, 2026. URL <https://arxiv.org/abs/2603.24621>.
- Google DeepMind. Gemini 3 pro model card, 2025. URL <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Tim Hesterberg. Weighted average importance sampling and defensive mixture distributions. *Technometrics*, 37(2):185–194, 1995.
- Valentin Hofmann, David Heineman, Ian Magnusson, Kyle Lo, Jesse Dodge, Maarten Sap, Pang Wei Koh, Chun Wang, Hannaneh Hajishirzi, and Noah A Smith. Fluid language model benchmarking. *arXiv preprint arXiv:2509.11106*, 2025.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 34303–34326. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/maia-polo24a.html>.
- Nestor Maslej, Loredana Fattorini, Raymond Perrault, Yolanda Gil, Vanessa Parli, Njenga Kariuki, Emily Capstick, Anka Reuel, Erik Brynjolfsson, John Etchemendy, Katrina Ligett, Terah Lyons, James Manyika, Juan Carlos Nieves, Yoav Shoham, Russell Wald, Toby Walsh, Armin Hamrah, Lapo Santarlasci, Julia Betts Lotufo, Alexandra Rome, Andrew Shi, and Sukrut Oak. Artificial intelligence index report 2025, 2025. URL <https://arxiv.org/abs/2504.07139>.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- METR. Task-completion time horizons of frontier ai models. <https://metr.org/time-horizons/>, 03 2026.

- Seyed Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. GSM-symbolic: Understanding the limitations of mathematical reasoning in large language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=AjXkRZIVjB>.
- OpenAI. Openai o1 system card, 2024. URL <https://openai.com/index/openai-o1-system-card/>.
- OpenAI. Introducing gpt-5.2, 2025. URL <https://openai.com/index/introducing-gpt-5-2/>.
- OpenAI. Gpt-5.4 thinking system card. <https://deploymentsafety.openai.com/gpt-5-4-thinking/gpt-5-4-thinking.pdf>, 2026.
- Yotam Perlitz, Elron Bandel, Ariel Gera, Ofir Arviv, Liat Ein-Dor, Eyal Shnarch, Noam Slonim, Michal Shmueli-Scheuer, and Leshem Choshen. Efficient benchmarking (of language models). In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 2519–2536, 2024.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and computing in applied probability*, 1(2):127–190, 1999.
- Reuven Y Rubinstein and Dirk P Kroese. *The cross-entropy method: a unified approach to combinatorial optimization, Monte-Carlo simulation, and machine learning*, volume 133. Springer, 2004.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *arXiv preprint arXiv:1907.10641*, 2019.
- Sang T. Truong, Yuheng Tu, Percy Liang, Bo Li, and Sanmi Koyejo. Reliable and efficient amortized model-based evaluation. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=HDbWrsgkB9>.
- Joshua Vendrow, Edward Vendrow, Sara Beery, and Aleksander Madry. Large language model benchmarks do not test reliability. In *Neurips Safe Generative AI Workshop 2024*, 2024. URL <https://openreview.net/forum?id=XSeN6xZtZ9>.
- Rajan Vivek, Kawin Ethayarajh, Diyi Yang, and Douwe Kiela. Anchor points: Benchmarking models with much fewer examples. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1576–1601, 2024.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In Tal Linzen, Grzegorz Chrupała, and Afra Alishahi (eds.), *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pp. 353–355, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-5446. URL <https://aclanthology.org/W18-5446/>.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Skyler Wu, Yash Nair, and Emmanuel J. Candès. Efficient evaluation of llm performance with statistical guarantees, 2026. URL <https://arxiv.org/abs/2601.20251>.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhui Chen. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *Proceedings of CVPR*, 2024.

Xiang Yue, Tianyu Zheng, Yuansheng Ni, Yubo Wang, Kai Zhang, Shengbang Tong, Yuxuan Sun, Botao Yu, Ge Zhang, Huan Sun, Yu Su, Wenhui Chen, and Graham Neubig. Mmmu-pro: A more robust multi-discipline multimodal understanding benchmark. In *Proceedings of ACL*, 2025.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

A LLM Inference Setup

We describe the implementation details for LLM inference. We use the same system prompt for all models:

Please reason step by step, and put your final answer within `\\boxed{}`.

and parse the final answer within the box. We follow the recommended decoding hyperparameters specified in each model card: temperature of 0.7 and top-p of 0.8 for Qwen2.5-Math-7B-Instruct, 1.0 and 1.0 for gpt-oss-20b-low, and 1.0 and 0.95 for Gemini 2.5 Flash Lite. For all models, we use a maximum of 32768 new tokens. If not specified, we use the default values.

B Algorithm for calculating confidence bounds

Algorithm 2 COMPUTEESTIMATE: Importance Sampling with Confidence Bounds

- 1: **Input:** Failure indicator f , distributions P and Q , sample size N , confidence level α
 - 2: **Output:** Estimate $\hat{\mu}$, confidence interval $[\hat{\mu}_{\text{low}}, \hat{\mu}_{\text{high}}]$
 - 3: Sample $Z_1, \dots, Z_N \sim Q$
 - 4: Compute importance-weighted estimate: $\hat{\mu} \leftarrow \frac{1}{N} \sum_{i=1}^N f(Z_i) \frac{P(Z_i)}{Q(Z_i)}$
 - 5: Compute sample variance: $\hat{\sigma}^2 \leftarrow \frac{1}{N-1} \sum_{i=1}^N \left(f(Z_i) \frac{P(Z_i)}{Q(Z_i)} - \hat{\mu} \right)^2$
 - 6: Set $Z_{\alpha/2} \leftarrow$ critical value from standard normal for confidence level α
 - 7: Compute CI: $[\hat{\mu}_{\text{low}}, \hat{\mu}_{\text{high}}] \leftarrow \left[\hat{\mu} - Z_{\alpha/2} \frac{\hat{\sigma}}{\sqrt{N}}, \hat{\mu} + Z_{\alpha/2} \frac{\hat{\sigma}}{\sqrt{N}} \right]$
 - 8: **Return** $\hat{\mu}, [\hat{\mu}_{\text{low}}, \hat{\mu}_{\text{high}}]$
-

Algorithm 2 details the COMPUTEESTIMATE subroutine called in Algorithm 1 (Section 5).

C Model-Template Pairs with Too Few Failures

The following model-template pairs have zero observed failures within our 100K evaluation budget to support reliable estimation (failure counts from the full 100K cache):

- gpt-oss-20b-low: Templates 1, 3, 5, 6 ($K = 8$)
- Gemini 2.5 Flash Lite: Templates 1, 3, 5, 7, 8 ($K = 8$)

We expect systematic failure patterns may emerge with a larger evaluation budget, and leave this for future work.

D Stronger Failure Concentration with Larger K

To supplement Table 1, we compare TV distance for each parameter dimension with increasing K . The distance is measured between the normalized failure distribution per parameter dimension and the uniform baseline; we report the three parameter dimensions with the largest distances. We observe that increasing K leads to larger distances across models and templates. This implies that with larger K , the failure distribution has shorter tails (fewer inconsistent failures) and probability mass concentrates more on the dominant failure-inducing inputs. This empirically supports the efficiency gains that grow with K in Table 2.

Qwen2.5-Math-7B-Instruct									
	$K = 4$			$K = 8$			$K = 16$		
Template	Top1	Top2	Top3	Top1	Top2	Top3	Top1	Top2	Top3
0	0.822	0.705	0.400	0.884	0.749	0.433	0.899	0.749	0.509
1	0.629	0.485	0.477	0.857	0.713	0.710	0.914	0.833	0.750
2	0.424	0.213	0.069	0.495	0.234	0.022	0.538	0.249	0.019
3	0.230	0.218	0.184	0.364	0.296	0.219	0.472	0.362	0.310
4	0.324	0.173	0.126	0.347	0.229	0.154	0.339	0.264	0.185
5	0.752	0.446	0.266	0.799	0.599	0.559	0.939	0.866	0.799
6	0.835	0.623	0.333	0.862	0.708	0.269	0.878	0.752	0.246
7	0.682	0.588	0.380	0.773	0.696	0.528	0.797	0.748	0.556
8	0.850	0.750	0.524	—	—	—	—	—	—

gpt-oss-20b-low									
	$K = 8$			$K = 16$			$K = 24$		
Template	Top1	Top2	Top3	Top1	Top2	Top3	Top1	Top2	Top3
0	0.708	0.621	0.394	0.900	0.800	0.695	0.933	0.850	0.729
2	0.550	0.390	0.041	0.648	0.459	0.053	0.699	0.541	0.052
4	0.419	0.292	0.223	0.552	0.396	0.335	0.631	0.508	0.385
7	0.694	0.412	0.390	0.836	0.555	0.468	0.895	0.633	0.537
8	0.680	0.541	0.300	0.732	0.590	0.413	0.760	0.599	0.487

Gemini 2.5 Flash Lite									
	$K = 8$			$K = 16$			$K = 24$		
Template	Top1	Top2	Top3	Top1	Top2	Top3	Top1	Top2	Top3
0	0.739	0.664	0.118	0.829	0.792	0.237	0.867	0.850	0.267
2	0.696	0.691	0.032	0.848	0.750	0.062	0.907	0.750	0.080
4	0.201	0.093	0.086	0.224	0.110	0.107	0.208	0.133	0.102
6	0.747	0.701	0.118	0.969	0.875	0.143	0.983	0.889	0.189

Table 3: Top-3 largest TV distances between the failure distribution and the uniform baseline, reported across models, templates, and majority-vote ensemble sizes K . — denotes model-templat pairs that have zero failures.

E Failure Histograms for All Parameters

Figure 6 shows parameter-wise failure histograms for all parameters, supplementing Figure 2 in Section 4. Parameters such as Name, Drinks, and Locations are roughly uniformly distributed among failures. In contrast, parameters such as n (first column), frac2 (second column), and x (third column) show strong concentration of failures at a few specific values.

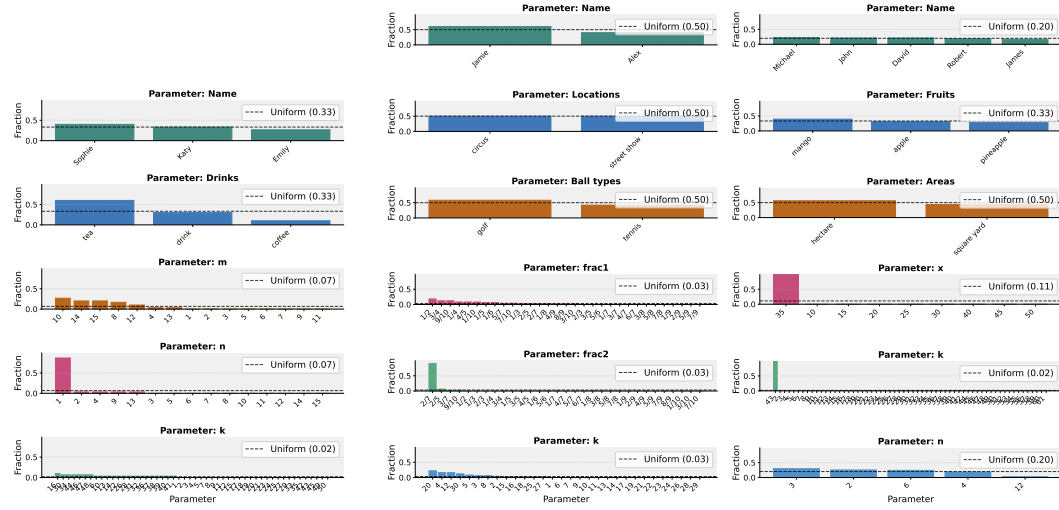


Figure 6: Parameter-wise failure histograms reveal strong concentration of errors.

F Table of Failure Probability Evaluation

This section provides the full table of estimated failure probabilities across models, templates, and K ($\lambda = 0.1$), corresponding to Figure 5.

ID	Estimate ($\hat{\mu}_Q$) $\pm$ Bounds	Estimate ($\hat{\mu}_Q$) $\pm$ Bounds	Estimate ($\hat{\mu}_Q$) $\pm$ Bounds
Qwen2.5-Math-7B-Instruct			
	$K=4$	$K=8$	$K=16$
0	$1.50\text{e-}03 \pm 4.90\text{e-}05$	$8.98\text{e-}04 \pm 3.03\text{e-}05$	$6.14\text{e-}04 \pm 1.50\text{e-}05$
1	$1.79\text{e-}03 \pm 9.20\text{e-}05$	$4.34\text{e-}04 \pm 3.36\text{e-}05$	$2.16\text{e-}04 \pm 1.79\text{e-}05$
2	$1.25\text{e-}01 \pm 6.97\text{e-}04$	$9.11\text{e-}02 \pm 5.73\text{e-}04$	$7.21\text{e-}02 \pm 4.98\text{e-}04$
3	$4.36\text{e-}02 \pm 4.87\text{e-}04$	$1.80\text{e-}02 \pm 2.95\text{e-}04$	$9.29\text{e-}03 \pm 1.94\text{e-}04$
4	$7.79\text{e-}02 \pm 6.22\text{e-}04$	$5.62\text{e-}02 \pm 5.21\text{e-}04$	$4.47\text{e-}02 \pm 4.61\text{e-}04$
5	$2.95\text{e-}03 \pm 9.20\text{e-}05$	$7.41\text{e-}04 \pm 3.29\text{e-}05$	$2.03\text{e-}04 \pm 1.62\text{e-}05$
6	$4.40\text{e-}03 \pm 5.02\text{e-}05$	$3.50\text{e-}03 \pm 4.16\text{e-}05$	$3.08\text{e-}03 \pm 3.48\text{e-}05$
7	$2.28\text{e-}03 \pm 6.37\text{e-}05$	$1.46\text{e-}03 \pm 3.50\text{e-}05$	$1.14\text{e-}03 \pm 2.13\text{e-}05$
8	$1.96\text{e-}04 \pm 2.36\text{e-}05$	$0.0, 5.30\text{e-}06$	$0.0, 5.30\text{e-}06$
gpt-oss-20b-low			
ID	$K=8$	$K=16$	$K=24$
0	$5.22\text{e-}04 \pm 3.40\text{e-}05$	$2.54\text{e-}04 \pm 1.90\text{e-}05$	$1.70\text{e-}04 \pm 1.19\text{e-}05$
2	$4.87\text{e-}02 \pm 3.55\text{e-}04$	$2.55\text{e-}02 \pm 2.42\text{e-}04$	$1.89\text{e-}02 \pm 1.98\text{e-}04$
4	$1.09\text{e-}02 \pm 2.24\text{e-}04$	$3.71\text{e-}03 \pm 1.15\text{e-}04$	$2.29\text{e-}03 \pm 7.96\text{e-}05$
7	$1.42\text{e-}03 \pm 6.18\text{e-}05$	$5.17\text{e-}04 \pm 2.70\text{e-}05$	$3.42\text{e-}04 \pm 1.92\text{e-}05$
8	$3.49\text{e-}03 \pm 7.65\text{e-}05$	$1.61\text{e-}03 \pm 4.93\text{e-}05$	$1.12\text{e-}03 \pm 3.86\text{e-}05$
Gemini 2.5 Flash Lite			
ID	$K=4$	$K=8$	$K=16$
0	$2.74\text{e-}03 \pm 1.21\text{e-}04$	$7.18\text{e-}04 \pm 3.63\text{e-}05$	$3.28\text{e-}04 \pm 1.65\text{e-}05$
2	$9.53\text{e-}03 \pm 1.66\text{e-}04$	$4.73\text{e-}03 \pm 7.29\text{e-}05$	$3.35\text{e-}03 \pm 4.06\text{e-}05$
4	$5.54\text{e-}02 \pm 5.69\text{e-}04$	$3.37\text{e-}02 \pm 4.42\text{e-}04$	$2.59\text{e-}02 \pm 3.87\text{e-}04$
6	$8.70\text{e-}03 \pm 2.22\text{e-}04$	$1.27\text{e-}03 \pm 6.26\text{e-}05$	$9.53\text{e-}04 \pm 1.37\text{e-}05$

Table 4: Estimated failure probability across templates, models, and K at $\lambda = 0.1$. For all missing model-template pairs are those that have zero failures within our evaluation budgets, and they can be estimated via exact binomial bounds of $(0, 5.30\text{e-}06)$.

G Parameterization of GSM8K Templates

Motivated by GSM-Symbolic (Mirzadeh et al., 2025), we construct parameterized GSM8K templates with extended variables and corrected grammatical flaws. Each template consists of a prompt with placeholders, a variable domain, and a formula determining the answer.

Template 0

- Prompt:
 $\{Z_{\text{Name}}\}$ saw a $\{Z_x\}$ -foot $\{Z_{\text{Fish}}\}$ with $\{Z_y\}$ remoras, each $\{Z_z\}$ -inches long, attached to it. What percentage of the $\{Z_{\text{Fish}}\}$'s body length is the combined length of the remoras?
- Variables:
 $Z_{\text{Name}} \in \{\text{Benny, Alex, Casey, Jordan, Taylor, Morgan, Riley, Sam, Jamie, Avery}\}$
 $Z_{\text{Fish}} \in \{\text{dolphin, whale, shark}\}$
 $Z_k \in \{2, \dots, 21\}$
 $Z_y \in \{1, \dots, 30\}$
 $Z_{\text{ratio}} \in \{4, 5, 10, 20, 25\}$
- Formula:
 $Z_y = Z_y \times 12$
 $Z_x = Z_{\text{ratio}} \times Z_k \times Z_y / 12$
 $\text{answer} = 100 / \text{ratio}$

Template 1

- Prompt:
A $\{Z_{\text{weather}}\}$ rolls in from the ocean to cover $\{Z_{\text{city}}\}$. It takes $\{Z_t\}$ minutes to cover every $\{Z_d\}$ miles of the city. If the city is $\{Z_y\}$ miles across from the oceanfront to the opposite inland edge, how many minutes will it take for the $\{Z_{\text{weather}}\}$ to cover the whole city?
- Variables:
 $Z_{\text{city}} \in \{\text{Harborview, Lakewood, Riverton}\}$
 $Z_{\text{weather}} \in \{\text{fog bank, marine layer, coastal mist}\}$
 $Z_t \in \{2, \dots, 71\}$
 $Z_d \in \{2, \dots, 13\}$
 $Z_s \in \{1, \dots, 12\}$
- Formula:
 $Z_y = Z_d \times Z_s$
 $\text{answer} = Z_s \times Z_t$

Template 2

- Prompt:
 $\{Z_{\text{name}}\}$ is playing a $\{Z_{\text{game}}\}$ during $\{Z_{\text{setting}}\}$ for a $\{Z_{\text{prize}}\}$. $\{Z_{\text{name}}\}$ is rolling a $\{Z_s\}$ -sided die. By how many percentage points is it more likely that they roll a number greater than $\{Z_{\text{target}}\}$ than that they roll two $\{Z_z\}$ numbers in a row?
- Variables:
 $Z_{\text{name}} \in \{\text{Jerry, Alex, Casey, Jordan, Taylor, Morgan}\}$
 $Z_{\text{setting}} \in \{\text{game night, casino night, board game meetup}\}$
 $Z_{\text{game}} \in \{\text{dice game}\}$
 $Z_{\text{prize}} \in \{\text{sticker, token, pin}\}$
 $Z_{\text{target}} \in \{2, 4, 6, \dots, 200\}$
 $Z_d \in \{2, 4, 5, 10, 20, 25, 50, 100\}$
 $Z_z \in \{\text{even, odd}\}$
- Formula:
 $Z_s = Z_{\text{target}} \times Z_d$
 $\text{answer} = \frac{Z_d - 1}{Z_d} \cdot 100 - 25$

Template 3

- Prompt:
A $\{Z_{\text{group}}\}$ of $\{Z_n\}$ students has various hobbies. $\{Z_{n1}\}$ like to $\{Z_{\text{hobby1}}\}$, $\{Z_{n2}\}$ like to play $\{Z_{\text{sport}}\}$, and the rest like to either $\{Z_{\text{hobby3}}\}$ or $\{Z_{\text{hobby4}}\}$. How many like to $\{Z_{\text{hobby3}}\}$ if the number that like to $\{Z_{\text{hobby4}}\}$ is $\{Z_{\text{multi}}\}$ times the number that prefer playing $\{Z_{\text{sport}}\}$?
- Variables:
 $Z_{\text{sport}} \in \{\text{basketball, soccer}\}$
 $Z_{\text{hobby1}} \in \{\text{read, paint}\}$
 $Z_{\text{hobby3}} \in \{\text{play music, garden}\}$
 $Z_{\text{hobby4}} \in \{\text{write}\}$
 $Z_{n1} \in \{2, \dots, 11\}$
 $Z_{n2} \in \{2, \dots, 11\}$
 $Z_{\text{extra}} \in \{1, \dots, 10\}$
 $Z_{\text{multi}} \in \{2, \dots, 11\}$
- Formula:
 $Z_n = Z_{n1} + (Z_{\text{multi}} + 1) \times Z_{n2} + Z_{\text{extra}}$
 $\text{answer} = Z_{\text{extra}}$

Template 4

- Prompt:
 $\{Z_{\text{name}}\}$ is popping popcorn for a snack. As the $\{Z_p\}$ of kernels heats up, the kernels start popping faster. $\{Z_n\}$ pop in the first $\{Z_s\}$ seconds of cooking, then $\{Z_{k2}\}$ times that amount in the next $\{Z_s\}$ seconds. The kernels increase to $\{Z_{k3}\}$ times the initial popping rate in the next $\{Z_s\}$ seconds, but in the final $\{Z_s\}$ seconds, the popping slows down to $\{Z_{r1}\}$ of the rate as the past $\{Z_s\}$ seconds. After $\{Z_{\text{name}}\}$ takes the $\{Z_p\}$ off the heat, a $\{Z_{r2}\}$ of the number of kernels that popped in the final $\{Z_s\}$ seconds of cooking also pop from the residual heat. How many pieces of popcorn does $\{Z_{\text{name}}\}$ have to eat?
- Variables:
 $Z_{\text{name}} \in \{\text{Garrett, Alex, Casey}\}$
 $Z_p \in \{\text{pan, pot, skillet}\}$
 $Z_{r1} \in \{\text{half, quarter}\}$
 $Z_{r2} \in \{\text{half, quarter}\}$
 $Z_n \in \{16, 32, 48, \dots, 160\}$
 $Z_s \in \{2, 3, 4\}$
 $Z_{k2} \in \{2, \dots, 11\}$
 $Z_{k3} \in \{12, \dots, 21\}$
- Formula:
 $\text{second} = Z_{k2} \cdot Z_n$
 $\text{third} = Z_{k3} \cdot Z_n$
 $\text{fourth} = \text{third} / Z_{r1\text{num}}$
 $\text{residual} = \text{third} / (Z_{r1\text{num}} \cdot Z_{r2\text{num}})$
 $\text{answer} = Z_n + \text{second} + \text{third} + \text{fourth} + \text{residual}$

Template 5

- Prompt:
 $\{Z_{\text{name}}\}$ makes $\{Z_{\text{drink}}\}$ using teaspoons of sugar and cups of water in the ratio of $\{Z_m\}:\{Z_n\}$. If she used a total of $\{Z_x\}$ teaspoons of sugar and cups of water, calculate the number of teaspoonfuls of sugar she used.
- Variables:
 $Z_{\text{name}} \in \{\text{Katy, Sophie, Emily}\}$
 $Z_{\text{drink}} \in \{\text{coffee, tea, drink}\}$
 $Z_m \in \{1, \dots, 15\}$
 $Z_n \in \{1, \dots, 15\}$
 $Z_k \in \{1, \dots, 50\}$
- Formula:
 $Z_{\text{total}} = Z_m + Z_n$
 $Z_x = Z_{\text{total}} \cdot Z_k$
 $\text{answer} = \frac{Z_m \cdot Z_x}{Z_{\text{total}}}$

Template 6

- Prompt:
 $\{Z_{\text{name}}\}$ has $\{Z_x\}$ $\{Z_{\text{area}}\}$ s of $\{Z_{\text{fruit}}\}$ s field. There are $\{Z_k\}$ $\{Z_{\text{fruit}}\}$ s per $\{Z_{\text{area}}\}$. $\{Z_{\text{name}}\}$ can harvest his $\{Z_{\text{fruit}}\}$ s every $\{Z_n\}$ months. How many $\{Z_{\text{fruit}}\}$ s can $\{Z_{\text{name}}\}$ harvest within a year?
- Variables:
 $Z_{\text{name}} \in \{\text{John, Michael, David, James, Robert}\}$
 $Z_{\text{fruit}} \in \{\text{pineapple, apple, mango}\}$
 $Z_{\text{area}} \in \{\text{hectare, square yard}\}$
 $Z_x \in \{10, 15, 20, \dots, 50\}$
 $Z_k \in \{2, \dots, 61\}$
 $Z_n \in \{2, 3, 4, 6, 12\}$
- Formula:
 $Z_{\text{cycles}} = 12 / Z_n$
 $\text{answer} = Z_x \cdot Z_k \cdot Z_{\text{cycles}}$

Template 7

- Prompt:
 $\{Z_{\text{name}}\}$ is juggling at a $\{Z_{\text{location}}\}$. $\{Z_{\text{name}}\}$ can juggle $\{Z_n\}$ balls. $\{Z_{\text{frac1}}\}$ of the balls are $\{Z_{\text{ball.type}}\}$ balls, and $\{Z_{\text{frac2}}\}$ of the $\{Z_{\text{ball.type}}\}$ balls are $\{Z_{\text{color}}\}$. How many $\{Z_{\text{color}}\}$ $\{Z_{\text{ball.type}}\}$ balls are there?
- Variables:
 $Z_{\text{name}} \in \{\text{Jamie, Alex}\}$
 $Z_{\text{location}} \in \{\text{circus, street show}\}$
 $Z_{\text{ball.type}} \in \{\text{golf, tennis}\}$
 $Z_{\text{color}} \in \{\text{blue}\}$
 $Z_{\text{frac1}}, Z_{\text{frac2}} \in \{1/2, 1/3, 2/3, 1/4, 3/4, 1/5, 2/5, 3/5, 4/5, 1/6, 5/6, 1/7, 2/7, 3/7, 4/7, 5/7, 6/7, 1/8, 3/8, 5/8, 7/8, 1/9, 2/9, 4/9, 5/9, 7/9, 8/9, 1/10, 3/10, 7/10, 9/10\}$
 $Z_k \in \{1, \dots, 30\}$
- Formula:
 $Z_n = Z_k \times \text{denom}(Z_{\text{frac1}}) \times \text{denom}(Z_{\text{frac2}})$
 $\text{answer} = Z_n \times (\text{num}(Z_{\text{frac1}}) \times \text{num}(Z_{\text{frac2}})) / (\text{denom}(Z_{\text{frac1}}) \times \text{denom}(Z_{\text{frac2}}))$

Template 8

- Prompt:
 $\{Z_{\text{name}}\}$ places $\{Z_{\text{obj}}\}$ s on some $\{Z_{\text{surface}}\}$ s. Each $\{Z_{\text{surface}}\}$ can hold $\{Z_x\}$ $\{Z_{\text{obj}}\}$ s. If he has $\{Z_n\}$ $\{Z_{\text{obj}}\}$ s and $\{Z_k\}$ $\{Z_{\text{surface}}\}$ s, how many $\{Z_{\text{obj}}\}$ s won't he be able to place on the $\{Z_{\text{surface}}\}$?
- Variables:
 $Z_{\text{name}} \in \{\text{Jaime, John, Mike}\}$
 $Z_{\text{obj}} \in \{\text{egg, olive, almond}\}$
 $Z_{\text{surface}} \in \{\text{tray, plate, table}\}$
 $Z_x \in \{2, \dots, 21\}$
 $Z_k \in \{2, \dots, 21\}$
 $Z_{\text{leftover}} \in \{1, \dots, 10\}$
- Formula:
 $Z_n = Z_k \cdot Z_x + Z_{\text{leftover}}$
 $\text{answer} = Z_{\text{leftover}}$

H CI Width vs. Number of Inferences

This section presents CI width versus number of inferences for all three models across varying templates and majority-vote sizes (K). Each figure corresponds to a specific model and K , with each column representing a different template. Within each subplot, the upper panel shows CI width vs. number of inferences, and the lower panel shows sorted CIs over 100 independent sample sets at the evaluation size of the bottom-right Pareto-frontier dot. Points with coverage in the range $99 \pm 1\%$ are shown as solid markers, while points with coverage $\geq 95\%$ are shown as semi-transparent markers. Efficiency gains are computed only for results achieving $99 \pm 1\%$ coverage.

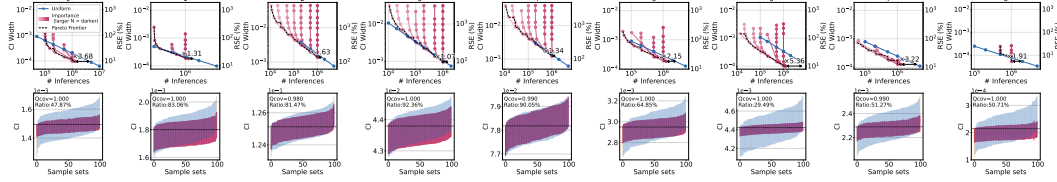


Figure 7: Qwen2.5-Math-7B-Instruct, $K = 4$, $\lambda = 0.1$

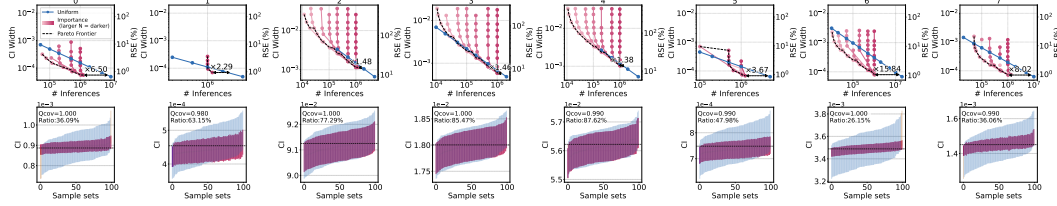


Figure 8: Qwen2.5-Math-7B-Instruct, $K = 8$, $\lambda = 0.1$

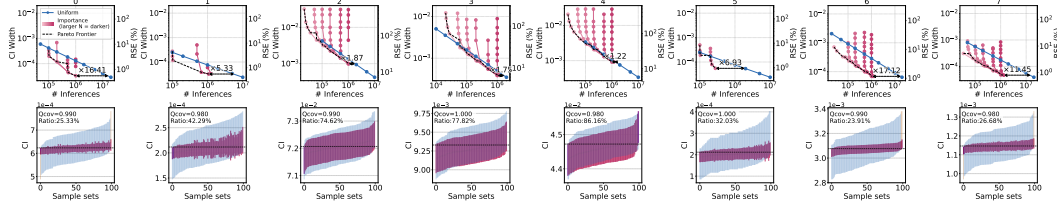
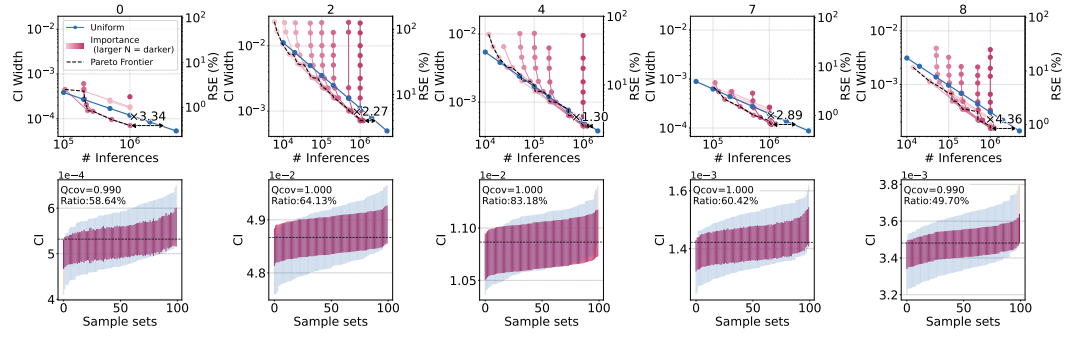
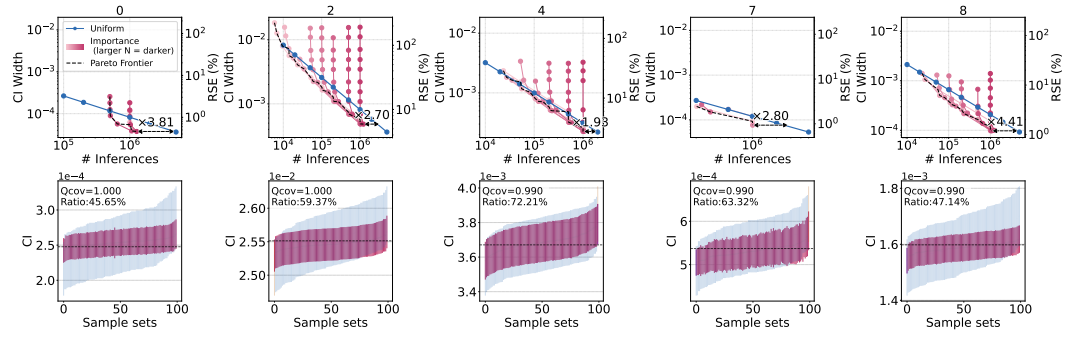
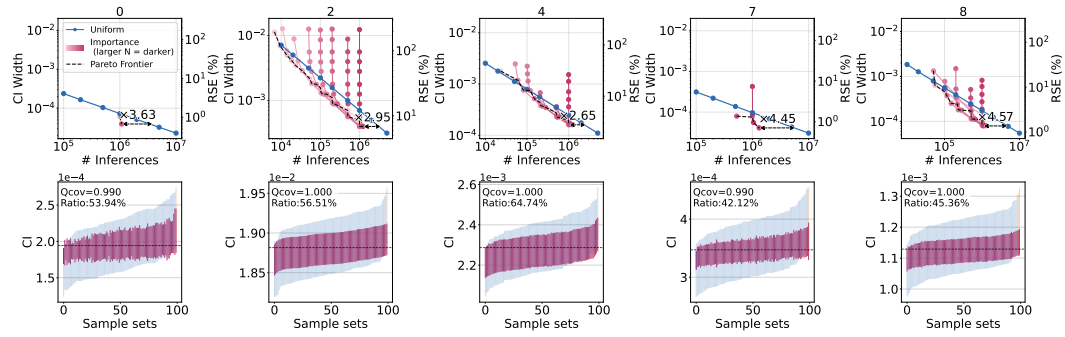
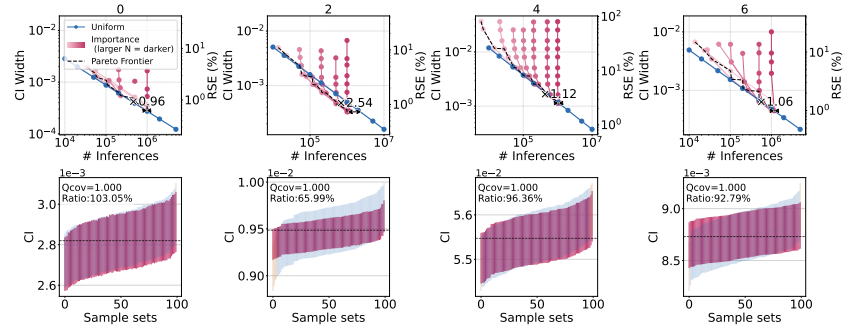
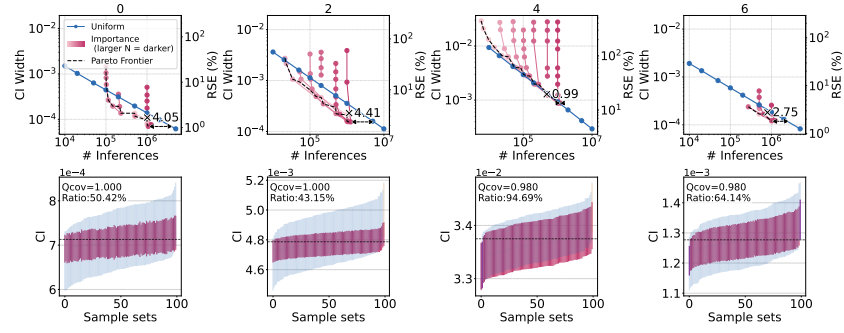
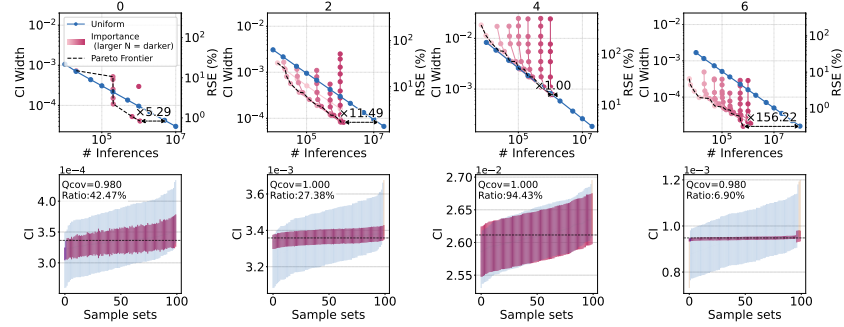


Figure 9: Qwen2.5-Math-7B-Instruct, $K = 16$, $\lambda = 0.1$

Figure 10: gpt-oss-20b-low, $K=8$, $\lambda=0.1$ Figure 11: gpt-oss-20b-low, $K=16$, $\lambda=0.1$ Figure 12: gpt-oss-20b-low, $K=24$, $\lambda=0.1$

Figure 13: Gemini 2.5 Flash Lite, $K = 4$, $\lambda = 0.1$ Figure 14: Gemini 2.5 Flash Lite, $K = 8$, $\lambda = 0.1$ Figure 15: Gemini 2.5 Flash Lite, $K = 16$, $\lambda = 0.1$

I Additional examples for repetitive failures by LLMs

To supplement the qualitative examples in Figure 3 (Section 4), we provide models’ responses that consistently produce erroneous outputs for certain prompts in Figures 19 and 24. The examples illustrate that Qwen2.5-Math-7B-Instruct exhibits repetitive errors for certain ratios of sugar teaspoons to water cups, frequently invoking a spurious “standard US measurement” heuristic to convert cups into tablespoons or teaspoons. For gpt-oss-20b-low on Template 7, the model shows striking failures on the basic arithmetic $84 \times (2/7)$, while being generally robust to other fraction values. We additionally provide dominant failure cases for all model-template pairs and observe various error types. These include simple arithmetic errors in addition/subtraction (18, 26), multiplication/division (16, 20, 17, 23, 25, 27), transcription (21), unnecessary approximation (22), and problem misinterpretation (24). Failure-inducing parameter values and the onset of erroneous reasoning in the generation traces are highlighted in red.

Prompt (Template 0)

A {dolphin} is {1276} feet long. {11} {remoras} attach to it, each measuring {348} inches in length. What percentage of the {dolphin}'s body length is the combined length of the {remoras}?

First Output (Qwen2.5-Math-7B-Instruct) (Incorrect)

To determine what percentage of the dolphin's body length is the combined length of the remoras, we need to follow these steps: ... Now, we perform the division:

$$\frac{319}{1276} \approx 0.2475$$

Then, we multiply by 100 to convert the decimal to a percentage:

$$0.2475 \times 100 = 24.75$$

Therefore, the combined length of the remoras is 24.75 percent of the dolphin's body length.

Second Output (Correct)

To determine what percentage of the dolphin's body length is the combined length of the remoras, we need to follow these steps: ... Perform the division and multiplication:

$$\frac{319}{1276} \approx 0.25$$

$$0.25 \times 100 = 25$$

Therefore, the combined length of the remoras is 25 percent of the dolphin's body length.

Third Output (Correct)

To determine what percentage of the dolphin's body length is the combined length of the remoras, we need to follow these steps: ...

$$\left(\frac{319}{1276} \right) \times 100 \approx 25 \text{ percent}$$

Thus, the combined length of the remoras is 25 percent of the dolphin's body length.

Fourth Output (Incorrect)

To determine what percentage of the dolphin's body length is the combined length of the remoras, we need to follow these steps: ... Now, we perform the division and multiplication:

$$\frac{319}{1276} \approx 0.247$$

$$0.247 \times 100 = 24.7$$

Therefore, the combined length of the remoras is approximately 24.7% of the dolphin's body length. The final answer is:

$$\boxed{24.7}$$

Fifth Output (Incorrect)

To determine what percentage of the dolphin's body length is the combined length of the remoras, we need to follow these steps: ...

$$\left(\frac{319 \text{ feet}}{1276 \text{ feet}} \right) \times 100 = \frac{31900}{1276} \approx 24.73\%$$

Therefore, the combined length of the remoras is 24.73 % of the dolphin's body length.

Sixth Output (Correct)

To determine what percentage of the dolphin's body length is the combined length of the remoras, we need to follow these steps: ...

$$\text{Percentage} = \left(\frac{319 \text{ feet}}{1276 \text{ feet}} \right) \times 100 \approx 25\%$$

Therefore, the combined length of the remoras is 25 % of the dolphin's body length.

Seventh Output (Incorrect)

To determine what percentage of the dolphin's body length is the combined length of the remoras, we need to follow these steps: ...

$$\text{Percentage} = \left(\frac{319 \text{ feet}}{1276 \text{ feet}} \right) \times 100 \approx 24.72\%$$

Therefore, the combined length of the remoras is 24.72 % of the dolphin's body length.

Eighth Output (Incorrect)

To determine what percentage of the dolphin's body length is the combined length of the remoras, we need to follow these steps: ... Perform the division and multiplication:

$$\frac{319}{1276} \approx 0.247$$

$$0.247 \times 100 = 24.7$$

Therefore, the combined length of the remoras is approximately 24.7 % of the dolphin's body length.

Answer:25

Figure 16: Eight independently generated outputs from Qwen2.5-Math-7B-Instruct evaluated on Template 0. Error type: wrong division.

Prompt (Template 1) A {coastal mist} rolls in from the ocean to cover {Harborview}. It takes {33} minutes to cover every {4} miles of the city. If the city is {28} miles across from the oceanfront to the opposite inland edge, how many minutes will it take for the {coastal mist} to cover the whole city?
First Output (Qwen2.5-Math-7B-Instruct) (Correct) To determine how many minutes it will take for the marine layer to cover the entire city, we start by identifying the rate at which the marine layer rolls in. The marine layer covers 4 miles in 33 minutes. We need to find out how many minutes it will take to cover 28 miles. ... $8.25 \times 28 = 8.25 \times (30 - 2) = 8.25 \times 30 - 8.25 \times 2$ $8.25 \times 30 = 247.5$ $8.25 \times 2 = 16.5$ $247.5 - 16.5 = 231$ Therefore, it will take the marine layer 231 minutes to cover the whole city.
Second Output (Incorrect) To determine how many minutes it will take for the marine layer to cover the entire city, we start by identifying the rate at which the marine layer rolls in. The marine layer covers 4 miles in 33 minutes. We need to find out how many minutes it will take to cover 28 miles. ... $\text{Time to cover 28 miles} = 8.25 \text{ minutes per mile} \times 28 \text{ miles} = 229.5 \text{ minutes}$ Therefore, it will take the marine layer 229.5 minutes to cover the whole city.
Third Output (Correct) To determine how many minutes it will take for the marine layer to cover the entire city, we need to follow these steps: ... Since it takes 33 minutes to cover each 4-mile segment, we multiply the number of segments by the time per segment: $\text{Total time} = 7 \text{ segments} \times 33 \text{ minutes/segment} = 231 \text{ minutes}$ Therefore, it will take 231 minutes for the marine layer to cover the whole city.
Fourth Output (Incorrect) To determine how many minutes it will take for the marine layer to cover the entire city, we start by identifying the rate at which the marine layer rolls in. The marine layer covers 4 miles in 33 minutes. We need to find out how long it will take to cover 28 miles. ... $8.25 \text{ minutes per mile} \times 28 \text{ miles} = 229.5 \text{ minutes}$ Therefore, it will take the marine layer 229.5 minutes to cover the whole city.
Fifth Output (Incorrect) To determine how many minutes it will take for the marine layer to cover the entire city, we start by finding the rate at which the marine layer rolls in. The marine layer covers 4 miles in 33 minutes. We need to find out how many minutes it takes to cover 28 miles. ... $8.25 \text{ minutes per mile} \times 28 \text{ miles} = 229.5 \text{ minutes}$ Therefore, it will take the marine layer 229.5 minutes to cover the whole city.
Sixth Output (Incorrect) To determine how many minutes it will take for the marine layer to cover the entire city, we start by identifying the rate at which the marine layer rolls in. The marine layer covers 4 miles in 33 minutes. We need to find out how many minutes it will take to cover 28 miles. ... $8.25 \text{ minutes per mile} \times 28 \text{ miles} = 229.5 \text{ minutes}$ Therefore, it will take the marine layer 229.5 minutes to cover the whole city.
Seventh Output (Correct) To determine how many minutes it will take for the marine layer to cover the entire city, we start by finding the rate at which the marine layer covers the city. The problem states that it takes 33 minutes to cover every 4 miles. ... Since it takes 33 minutes to cover each 4-mile segment, we multiply the number of segments by the time it takes to cover each segment: $\text{Total time} = 7 \text{ segments} \times 33 \text{ minutes/segment} = 231 \text{ minutes}$ Thus, the total time it will take for the marine layer to cover the whole city is: 231
Eighth Output (Incorrect) To determine how many minutes it will take for the marine layer to cover the entire city, we start by identifying the rate at which the marine layer rolls in. The marine layer covers 4 miles in 33 minutes. We need to find out how many minutes it will take to cover 28 miles. ... $8.25 \text{ minutes per mile} \times 28 \text{ miles} = 229.5 \text{ minutes}$ Therefore, it will take the marine layer 229.5 minutes to cover the whole city. Answer:231

Figure 17: Eight independently generated outputs from Qwen2.5-Math-7B-Instruct evaluated on Template 1. Error type: wrong multiplication.

Prompt (Template 3)

A group of {71} students has various hobbies. {4} like to read, {6} like to play basketball, and the rest like to either play music or write. How many like to play music if the number that like to write is {10} times the number that prefer playing basketball?

First Output (Qwen2.5-Math-7B-Instruct) (Incorrect)

To determine how many students like to play music, we need to follow these steps: ...

$$\text{Total} = 4 + 6 + 60 = 64$$

$$\text{Number of students who like to play music} = 71 - 64 = 7$$

Thus, the number of students who like to play music is 7.

Second Output (Incorrect)

To determine how many students like to play music, we start by identifying the number of students who like to write. According to the problem, the number of students who like to write is 10 times the number of students who like to play basketball. ...

$$71 - 4 - 60 = 7$$

Thus, the number of students who like to play music is 7.

Third Output (Correct)

To determine how many students like to play music, we start by identifying the number of students who like to write. ... Next, let m represent the number of students who like to play music. Then:

$$4 + 6 + m + 60 = 71$$

$$70 + m = 71$$

$$m = 1$$

Therefore, the number of students who like to play music is 1.

Fourth Output (Incorrect)

To determine how many students like to play music, we need to follow these steps: ...

$$\text{Total} = 4 + 6 + 60 = 66$$

$$\text{Number of students who like to play music} = 71 - 66 = 5$$

Thus, the number of students who like to play music is 5.

Fifth Output (Incorrect)

To determine how many students like to play music, we start by identifying the number of students who like to write. ...

$$4 + 6 + 60 = 66$$

Since the total number of students is 71, the number of students who like to play music is:

$$71 - 66 = 5$$

Thus, the number of students who like to play music is:

5

Sixth Output (Incorrect)

To determine how many students like to play music, we need to follow these steps: ...

$$\text{Total} = 4 + 6 + 60 = 60$$

$$\text{Number of students who like to play music} = 71 - 60 = 11$$

Thus, the number of students who like to play music is 11.

Seventh Output (Correct)

To determine how many students like to play music, we start by identifying the number of students who like to write. ...

$$71 - (4 + 6 + 60) = 71 - 70 = 1$$

Thus, the number of students who like to play music is 1.

Eighth Output (Incorrect)

To determine how many students like to play music, we need to follow these steps: ...

$$\text{Total} = 4 + 6 + 60 = 66$$

$$\text{Number of students who like to play music} = 71 - 66 = 5$$

Thus, the number of students who like to play music is 5.

Answer:1

Figure 18: Eight independently generated outputs from Qwen2.5-Math-7B-Instruct evaluated on Template 3. Error type: wrong addition/subtraction.

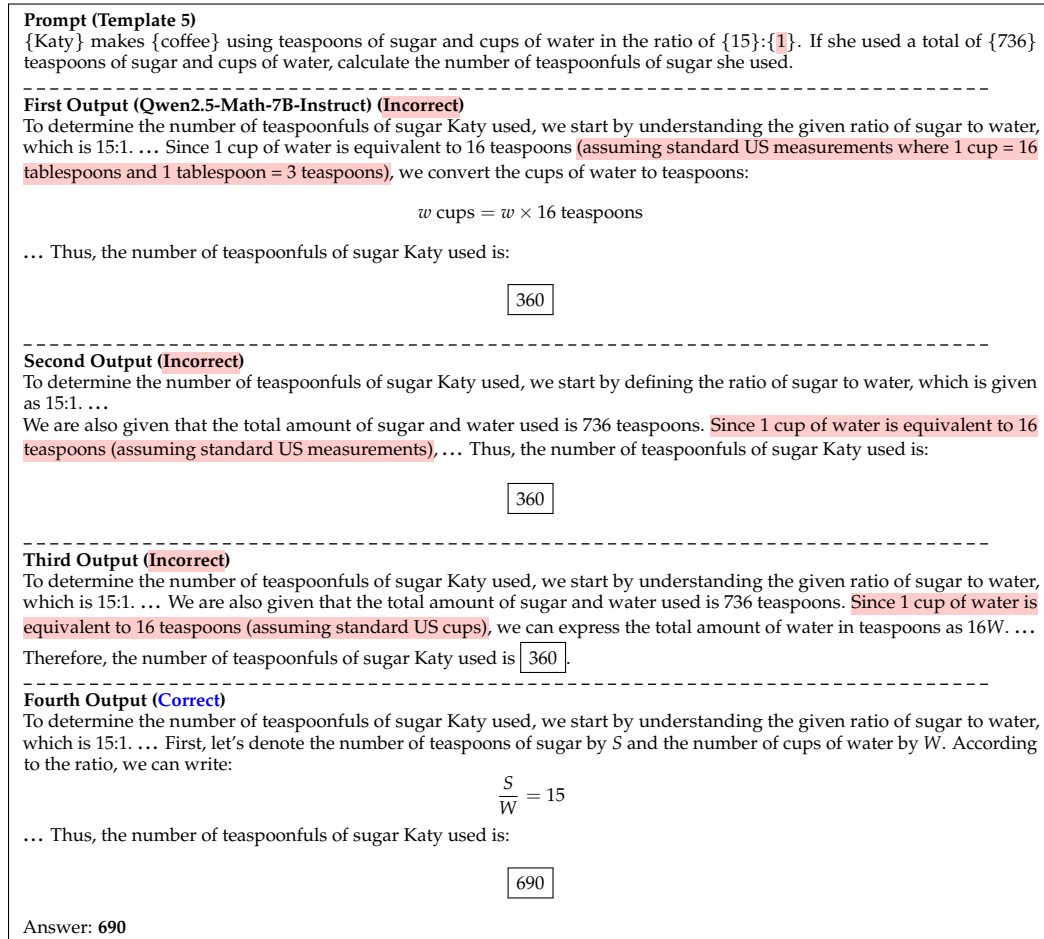


Figure 19: First four of eight independently generated outputs from Qwen2.5-Math-7B-Instruct evaluated on Template 5. Error type: unjustified unit conversion.

Prompt (Template 6) {John} has {15} {hectares} of {pineapples} field. There are {47} {pineapples} per {hectare}. {John} can harvest his {pineapples} every {2} months. How many {pineapples} can {John} harvest within a year?
First Output (Qwen2.5-Math-7B-Instruct) (Incorrect) To determine how many pineapples John can harvest within a year, we need to follow these steps: ... Total number of pineapples = 15 hectares $\times$ 47 pineapples/hectare = 655 pineapples Therefore, the total number of pineapples John can harvest within a year is 3930.
Second Output (Incorrect) To determine how many pineapples John can harvest within a year, we need to follow these steps: ... Total number of pineapples = 15 hectares $\times$ 47 pineapples/hectare = 695 pineapples Therefore, the total number of pineapples John can harvest within a year is 4170.
Third Output (Incorrect) To determine how many pineapples John can harvest within a year, we need to follow these steps: ... 15 hectares $\times$ 47 pineapples/hectare = 655 pineapples Therefore, the number of pineapples John can harvest within a year is 3930.
Fourth Output (Incorrect) To determine how many pineapples John can harvest within a year, we need to follow these steps: ... 15 hectares $\times$ 47 pineapples/hectare = 655 pineapples Therefore, the total number of pineapples John can harvest within a year is: 3930
Fifth Output (Incorrect) To determine how many pineapples John can harvest within a year, we need to follow these steps: ... Total number of pineapples = 15 hectares $\times$ 47 pineapples/hectare = 695 pineapples Therefore, the number of pineapples John can harvest within a year is 4170.
Sixth Output (Incorrect) To determine how many pineapples John can harvest within a year, we need to follow these steps: ... Total number of pineapples = 15 hectares $\times$ 47 pineapples/hectare = 655 pineapples Therefore, the total number of pineapples John can harvest within a year is 3930.
Seventh Output (Correct) To determine how many pineapples John can harvest within a year, we need to follow these steps: ... 15 hectares $\times$ 47 pineapples/hectare = 705 pineapples Therefore, the number of pineapples John can harvest within a year is 4230. Answer: 4230

Figure 20: Seven of eight independently generated outputs from Qwen2.5-Math-7B-Instruct evaluated on Template 6. Error type: wrong multiplication.

Prompt (Template 0)

{Benny} saw a {1920}-foot whale with {12} remoras, each {192} inches long, attached to it. What percentage of the whale's body length is the combined length of the remoras?

First Output (Qwen2.5-Math-7B-Instruct) (Incorrect)

analysisWe need combined length of remoras: $12 * 192$ inches. Convert to feet: 192 inches = 16 feet. So each 16 ft, $12 * 16 = 192$ ft. **Whale length 192 ft.** Percentage = $(192/192) * 100 = 100\%$ The whale's body length is given as 192 feet. Thus, the combined remora length equals the whale's length:

$$\frac{192 \text{ ft}}{192 \text{ ft}} = 1 \implies 100\%$$

100%

Second Output (Correct)

analysisCompute remoras total length: $12 * 192$ inches = 2304 inches. Convert whale length: 1920 ft = $1920 * 12 = 23040$ inches. Percentage = $2304 / 23040 = 0.1 = 10\%$ Thus the percentage is:

$$\frac{2304}{23040} \times 100\% = 10\%$$

10%

Third Output (Correct)

analysisNeed remora combined length: $12 * 192$ inches = 2304 inches = 192 inches? Wait 192 inches = 16 ft. $12 * 16 = 192$ ft. Whale length 1920 ft. Percentage = $192 / 1920 = 0.1 = 10\%$ As a percentage,

$$0.10 \times 100\% = 10\%$$

10%

Fourth Output (Incorrect)

analysisConvert remora length to feet: 192 inches / 12 = 16 ft. 12 remoras $\rightarrow 192$ ft. **Whale 192 ft.** Percentage = $192 / 192 * 100 = 100\%$

100%

Fifth Output (Incorrect)

analysisWe need combined length of remoras: $12 * 192$ inches. Convert to feet: 192 inches = 16 feet (since 12 inches = 1 ft, $192 / 12 = 16$). So each remora 16 ft. $12 * 16 = 192$ ft. **Whale length 192 ft.** So percentage = $(192 / 192) * 100 = 100\%$. Box. ...

$$\text{Combined remora length} = 12 \times 192 \text{ in} = 12 \times 16 \text{ ft} = 192 \text{ ft}$$

Whale length = 192 ft.

$$\text{Percentage} = \frac{192}{192} \times 100\% = 100\%$$

100%

Sixth Output (Correct)

analysisCompute total remora length: $12 * 192$ inches. Convert to feet: $12 * 192 = 2304$ inches. 12 feet? Wait $2304 / 12 = 192$ feet. Whale length 1920 feet. Percentage = $192 / 1920 * 100 = 10\%$ The whale is 1920 ft long. The percentage of the whale's length that the remoras occupy is:

$$\frac{192 \text{ ft}}{1920 \text{ ft}} \times 100\% = 10\%$$

10%

Seventh Output (Incorrect)

analysisConvert remora length to feet: 192 in = 16 ft. Each 16 ft. 12 remoras total $12 * 16 = 192$ ft. **Whale length 192 ft.** Percentage = $192 / 192 * 100 = 100\%$ The whale is also 192 ft long, so

$$\frac{192 \text{ ft}}{192 \text{ ft}} \times 100\% = 100\%$$

100%

Eighth Output (Incorrect)

analysisCompute remora total length: $12 * 192$ inches. Convert to feet: inches / 12. $192 \text{ in} = 16 \text{ ft}$. $12 * 16 = 192 \text{ ft}$. **Whale length 192 ft.** Ratio $192 / 192 = 1 \rightarrow 100\%$ The whale is 192 feet long, so the combined remora length equals the whale's length.

$$\frac{192 \text{ ft}}{192 \text{ ft}} = 1 \implies 100\%$$

100%

Answer: 10

Figure 21: Eight independently generated outputs from gpt-oss-20b-low evaluated on Template 0. Error type: transcription.

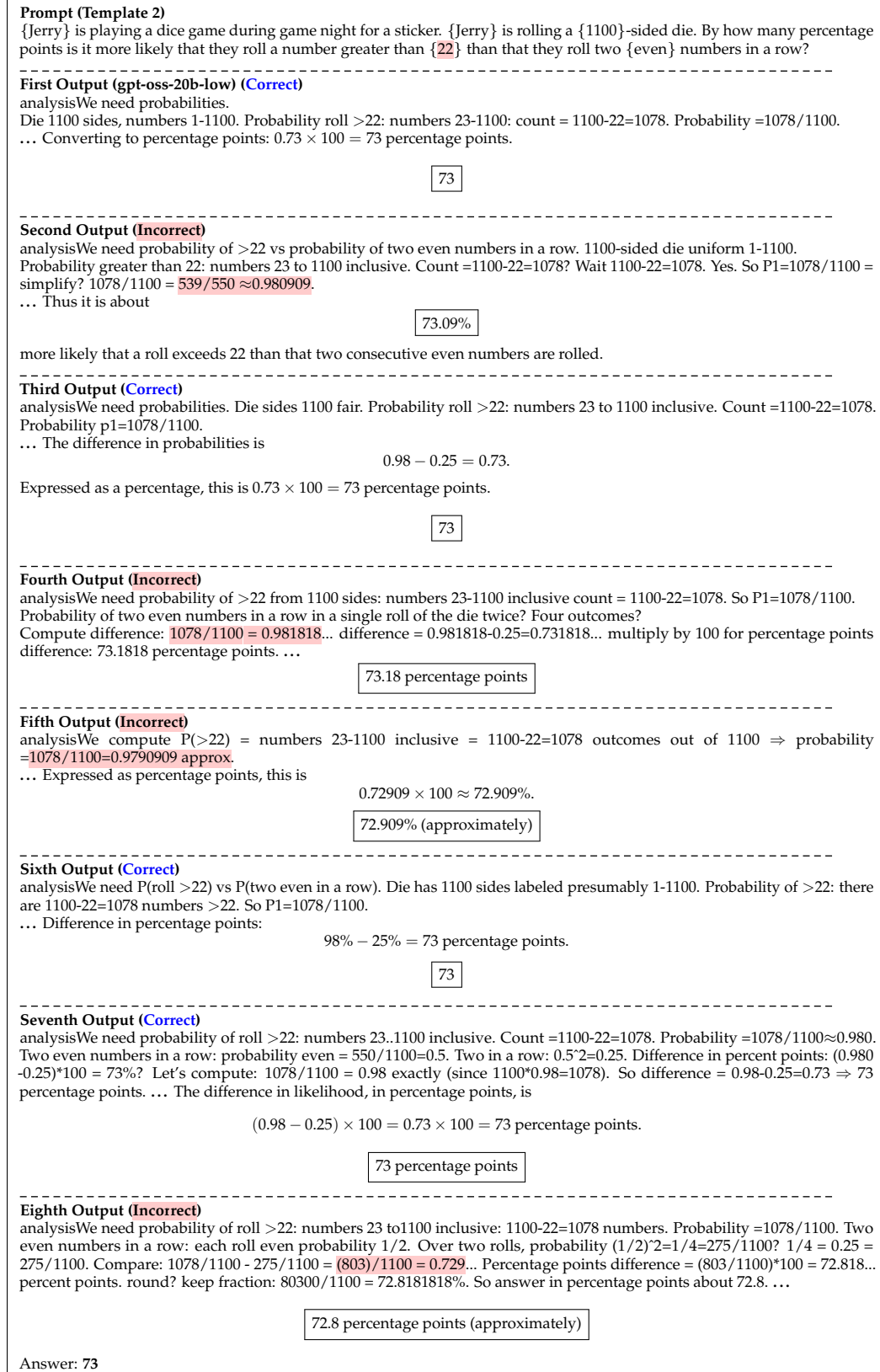


Figure 22: Eight independently generated outputs from gpt-oss-20b-low evaluated on Template 2. Error type: unnecessary approximation.

Prompt (Template 7) {Jamie} is juggling at a circus. {Jamie} can juggle {928} balls. {3/4} of the balls are golf balls, and {7/8} of the golf balls are blue. How many blue golf balls are there?
First Output (gpt-oss-20b-low) (Correct) To determine the number of blue golf balls Jamie has, we need to follow these steps: ... $696 \times \frac{7}{8} = 696 \div 8 \times 7 = 87 \times 7 = 609$ So, Jamie has 609 blue golf balls. Therefore, the number of blue golf balls is 609.
Second Output (Incorrect) To determine the number of blue golf balls Jamie has, we need to follow these steps: ... $696 \times \frac{7}{8} = 696 \times 0.875 = 612$ So, Jamie has 612 blue golf balls. The final answer is: 612
Third Output (Incorrect) To determine the number of blue golf balls Jamie is juggling, we need to follow these steps: ... $696 \times \frac{7}{8} = 696 \times 0.875 = 612$ So, Jamie has 612 blue golf balls. Therefore, the number of blue golf balls is 612.
Fourth Output (Incorrect) To determine the number of blue golf balls Jamie is juggling, we need to follow these steps: ... $696 \times \frac{7}{8} = 696 \times 0.875 = 612$ So, Jamie has 612 blue golf balls. The number of blue golf balls is 612.
Fifth Output (Correct) To determine the number of blue golf balls Jamie is juggling, we need to follow these steps: ... $696 \times \frac{7}{8} = 696 \div 8 \times 7 = 87 \times 7 = 609$ So, Jamie has 609 blue golf balls. The final answer is: 609
Sixth Output (Incorrect) To determine the number of blue golf balls Jamie has, we need to follow these steps: ... $696 \times \frac{7}{8} = 696 \times 0.875 = 612$ Therefore, Jamie has 612 blue golf balls. The final answer is: 612
Seventh Output (Correct) To determine the number of blue golf balls Jamie is juggling, we need to follow these steps: ... $7 \times 87 = 609$ So, there are 609 blue golf balls. Therefore, the number of blue golf balls is 609. Answer: 609

Figure 23: Seven of eight independently generated outputs from gpt-oss-20b-low evaluated on Template 7. Error type: wrong multiplication.

Prompt (Template 8) {Jaime} places eggs on some tables. Each table can hold {7} eggs. If he has {57} eggs and {7} tables, how many eggs won't he be able to place on the table? First Output (gpt-oss-20b-low) (Incorrect) analysisWe compute $57 \bmod 7$. $7 \times 8 = 56$, remainder 1. So answer 1 egg. 1 Second Output (Incorrect) analysisWe need remainder 57 mod 7. $7 \times 8 = 56$, remainder 1. So 1 egg can't be placed. 1 Third Output (Correct) analysisWe need to see: 7 tables each hold 7 eggs $\Rightarrow$ max capacity 49? Wait 7 tables $\times$ 7 eggs = 49. He has 57 eggs. So leftover = $57 - 49 = 8$. So answer 8. 1. Capacity of the tables: Each table holds 7 eggs. With 7 tables, the total capacity is $7 \text{ tables} \times 7 \text{ eggs/table} = 49 \text{ eggs.}$ 2. Eggs he has: Jaime has 57 eggs. 3. Eggs that cannot be placed: Subtract the capacity from the total eggs: $57 - 49 = 8$ 8 Fourth Output (Incorrect) analysisCompute $57 \bmod 7$. $7 \times 8 = 56$ remainder 1. 1 Fifth Output (Incorrect) analysisWe need remainder of 57 divided by 7. $7 \times 8 = 56$, remainder 1. So 1 egg won't fit. The number of eggs that cannot be placed is the remainder when 57 is divided by 7. $57 \div 7 = 8$ with a remainder of 1. So 1 egg cannot be placed on the tables. Sixth Output (Correct) analysisWe need 57 eggs, each table holds 7, 7 tables. Max capacity $7 \times 7 = 49$. Eggs unused = $57 - 49 = 8$. So answer 8. 8 Seventh Output (Incorrect) analysisWe compute $57 \bmod 7$. $7 \times 8 = 56$, remainder 1. So 1 egg left. To find the number of eggs that cannot be placed, divide the total eggs by the capacity of each table and look at the remainder: $57 \div 7 = 8$ with a remainder of 1 So after using 8 tables for 56 eggs, there is 1 egg that cannot be placed. 1 Eighth Output (Incorrect) analysisCompute $57 \bmod 7$. $7 \times 8 = 56$ remainder 1. So 1 egg cannot be placed. 1
Answer: 8

Figure 24: Eight generated outputs from gpt-oss-20b-low evaluated on Template 8. Error type: problem misinterpretation.

Prompt (Template 0) A dolphin is {2090} feet long. {19} remoras, each {264} inches long, attach to it. What percentage of the dolphin's body length is the combined length of the remoras? First Output (Qwen2.5-Math-7B-Instruct) (Incorrect) Here's how to solve this problem step-by-step: ... Combined length of remoras in inches: 19 remoras * 264 inches/remora = 4960 + 792 = 4960 + 800 - 8 = 5760 - 8 = 5752 inches 2. Calculate the combined length of the remoras as a percentage of the dolphin's length. ... Answer: The combined length of the remoras is approximately 22.93% of the dolphin's body length. Second Output (Incorrect) Here's how to solve this problem step-by-step: ... 2. Calculate the combined length of the remoras. - Number of remoras = 19 - Length of each remora = 264 inches - Combined length of remoras = 19 remoras * 264 inches/remora = 4956 inches ... The final answer is 19.8%. Third Output (Incorrect) Here's how to solve this problem step-by-step: 2. Calculate the combined length of the remoras. - Combined remora length = 19 remoras * 264 inches/remora = 4996 inches The final answer is 19.92%. Fourth Output (Incorrect) Here's how to solve this problem step-by-step: 1. Convert all measurements to the same unit. It's easiest to work with inches since the remoras are already measured in inches. ... 2. Calculate the combined length of the remoras. - There are 19 remoras. 19 remoras * 264 inches/remora = 4968 inches ... The final answer is 19.81%. Fifth Output (Correct) Here's how to solve this problem: ... 5. Calculate the Combined Length of the Remoras: Combined remora length = 19×264 inches = 5016 inches ... 9. Final Answer: The combined length of the remoras is 20% of the dolphin's body length. The final answer is 20%. Sixth Output (Incorrect) Here's how to solve this problem: ... 2. Calculate the combined length of the remoras. - Number of remoras: 19 - Length of each remora: 264 inches - Combined remora length: 19 remoras * 264 inches/remora = 4996 inches ... The final answer is 19.9%. Seventh Output (Incorrect) Here's how to solve this problem step-by-step: ... 2. Calculate the combined length of the remoras. - There are 19 remoras, and each is 264 inches long. - Combined remora length = 19 remoras * 264 inches/remora = 4916 inches ... The final answer is 19.6%. Eighth Output (Incorrect) Here's how to solve this problem step-by-step: ... 2. Calculate the combined length of the remoras. There are 19 remoras, and each is 264 inches long. - Combined remora length = 19 remoras * 264 inches/remora = 4916 inches ... The final answer is 19.6%. Answer: 20
--

Figure 25: Eight generated outputs from Gemini 2.5 Flash Lite evaluated on Template 0. Error type: wrong multiplication.

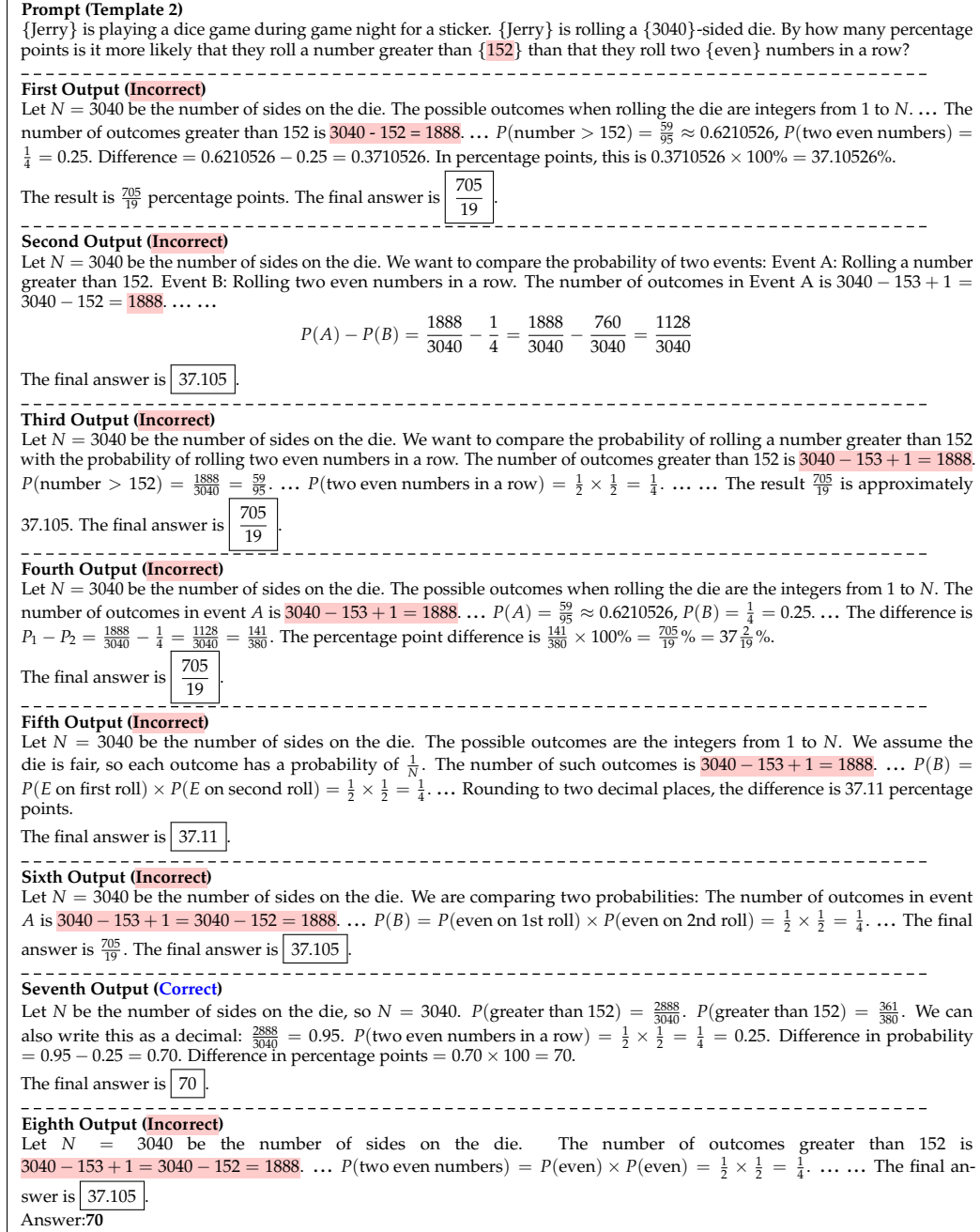


Figure 26: Eight generated outputs from Gemini 2.5 Flash Lite evaluated on Template 2. Error type: wrong subtraction.

Prompt (Template 6) {John} has {35} {square yard}s of {pineapple}s field. There are {43} {pineapple}s per {square yard}. {John} can harvest his {pineapple}s every {2} months. How many {pineapple}s can {John} harvest within a year?"
First Output (Gemini 2.5 Flash Lite) (Correct) Here's how to solve the problem step-by-step: 1. Calculate the total number of pineapples: * John has 35 square yards of field. * There are 43 pineapples per square yard. * Total pineapples = 35 square yards * 43 pineapples/square yard = 1505 pineapples. ... The final answer is 9030.
Second Output (Incorrect) Here's how to solve the problem step-by-step: 1. Calculate the total number of pineapples: * John has 35 square yards of field. * There are 43 pineapples per square yard. * Total pineapples = 35 square yards * 43 pineapples/square yard * Total pineapples = 1455 pineapples ... The final answer is 8730.
Third Output (Incorrect) Here's how to solve the problem step-by-step: 1. Calculate the total number of pineapples John has: * He has 35 square yards. * There are 43 pineapples per square yard. * Total pineapples = 35 square yards * 43 pineapples/square yard * 35 * 43 = 1495 pineapples ... The final answer is 8970.
Fourth Output (Incorrect) Here's how to solve the problem step-by-step: 1. Calculate the total number of pineapples: * John has 35 square yards of field. * There are 43 pineapples per square yard. * Total pineapples = 35 square yards * 43 pineapples/square yard = 1495 pineapples. ... The final answer is 8970.
Fifth Output (Incorrect) Here's how to solve the problem step-by-step: 1. Calculate the total number of pineapples: * John has 35 square yards of field. * There are 43 pineapples per square yard. * Total pineapples = 35 square yards * 43 pineapples/square yard = 1495 pineapples. ... The final answer is 8970. Answer: 9030

Figure 27: First five of eight independently generated outputs from Gemini 2.5 Flash Lite evaluated on Template 6. Error type: wrong multiplication.